



Tecnología de Base de Datos – Ingeniería en informática

“Trabajo Final Integrador”

Tema: Diseño e implementación de una base de datos.

Grupo: 2.

Integrantes:

NOMBRE	MATRICULA	CARRERA
Marinoni, Gino	67237	Ing. en informática
Meier, Andres Alberto	66772	Ing. en informática
Micheloni, Gianfranco	67154	Ing. en informática
Silvero Villagra, Lisandro Manuel	66428	Ing. en informática
Suligoy, Rodrigo	67144	Ing. en informática

Docentes: Ing. Barreyro, Enrique Luis.
Ing. Suenaga, Roberto.

Índice:

Etapa I.....	5
DER.....	5
Funcionalidades.....	6
Matriz funcionalidad, tabla y justificación.....	8
DDL (MySQL).....	12
Etapa II.....	23
Esquema.....	24
Secuencias.....	24
Insert Iniciales.....	24
Funciones.....	24
Índices.....	24
Procedimientos.....	24
Esquema Logs.....	24
Triggers.....	24
Jobs.....	24
ANEXO.....	32
DDL para PostgreSQL.....	32
Funciones.....	44
Procedimientos.....	52
Sucursal.....	52
Cliente.....	60
Tipo Vehículo.....	69
Taller.....	74
Vehículo.....	83
Mantenimiento.....	95
Reserva.....	111
Alquiler.....	127
Tarifa.....	142
Finalizar Alquiler y Migrar desde reserva.....	153
Secuencias.....	169
Triggers.....	171
Esquema Logs.....	256
Jobs.....	264

Consigna

TFI Tecnología de Base de Datos

Introducción:

En el marco de la materia, se desarrollará un Trabajo Final Integrador (TFI) cuyo objetivo es aplicar de manera práctica los conceptos vistos durante el cursado, abordando inicialmente el diseño y posterior implementación de una base de datos que resuelva una problemática real.

El trabajo se organizará en distintas etapas, permitiendo una evolución progresiva del sistema, incorporando nuevos requerimientos y complejidades a lo largo del desarrollo.

Modalidad de Trabajo:

El Trabajo Integrador deberá realizarse en grupos de cinco (5) alumnos.

Se espera que todos los integrantes participen activamente en el análisis, diseño y desarrollo de la solución propuesta.

Rol del Docente:

El docente actuará como cliente a satisfacer.

Por lo tanto:

- Toda consulta sobre el entendimiento de la lógica del sistema deberá ser realizada al docente.
- La propuesta de diseño deberá ser aprobada por el cliente antes de su implementación en la base de datos.

Comunicación Equipo de IT y Cliente:

El servidor discord ya definido. En el mismo una vez establecidos los grupos, se crearán grupos privados, en el cual se podrá plantear las dudas que surjan, así como validaciones sobre el desarrollo del trabajo.

Etapas:

- 1) Modelado de la Base de datos que solucione el problema (30/04/2026)
- 2) Implementación en el SGBD elegido, más lógica de negocio parcial aplicada en procedimientos / funciones / disparadores, etc. (21/05/2026)
- 3) Presentación de la solución final, lógica de negocio terminada en el SGBD más desarrollado en framework a elección, de las interfaces necesarias para el funcionamiento del sistema. (11/06/2026)

IMPORTANTE: La implementación en framework no abarcará la totalidad del sistema. Los módulos a desarrollar serán definidos durante el cursado.

Escenario 2026



Una empresa de alquiler de vehículos posee varias sucursales, en las cuales dispone de autos para alquilar.

Los clientes pueden alquilar vehículos por un período determinado, indicando la fecha de inicio y la fecha de devolución prevista.

Al momento de iniciar el alquiler, se debe registrar la cantidad de kilómetros del vehículo, y al momento de finalizarlo, se debe registrar nuevamente esta información.

Cada vehículo pertenece a una sucursal y tiene un tipo (por ejemplo: cupé, sedan, suv, etc.), además de información descriptiva como marca, modelo y un detalle de confort.

Por cada vehículo se podrán registrar entre 1 y 5 imágenes que permitan al cliente visualizar sus características.

Los vehículos pueden estar en distintos estados, tales como: disponible, alquilado o en mantenimiento.

Cada alquiler corresponde a un único cliente y a un único vehículo.

El valor del alquiler es por día, y depende tanto del tipo de vehículo como de la sucursal donde se realiza el alquiler.

Asimismo, en caso de que el vehículo sea devuelto fuera del tiempo pactado, se deberá cobrar un recargo por hora excedida, calculado como un porcentaje del valor diario del alquiler. Dicho porcentaje también depende del tipo de vehículo y de la sucursal.

El sistema deberá generar una factura al finalizar el alquiler a nombre del cliente, donde se detalle el costo del alquiler y los recargos aplicados.

Los clientes podrán realizar reservas de vehículos de forma online, para lo cual deberán registrarse en el sistema mediante un usuario y contraseña.

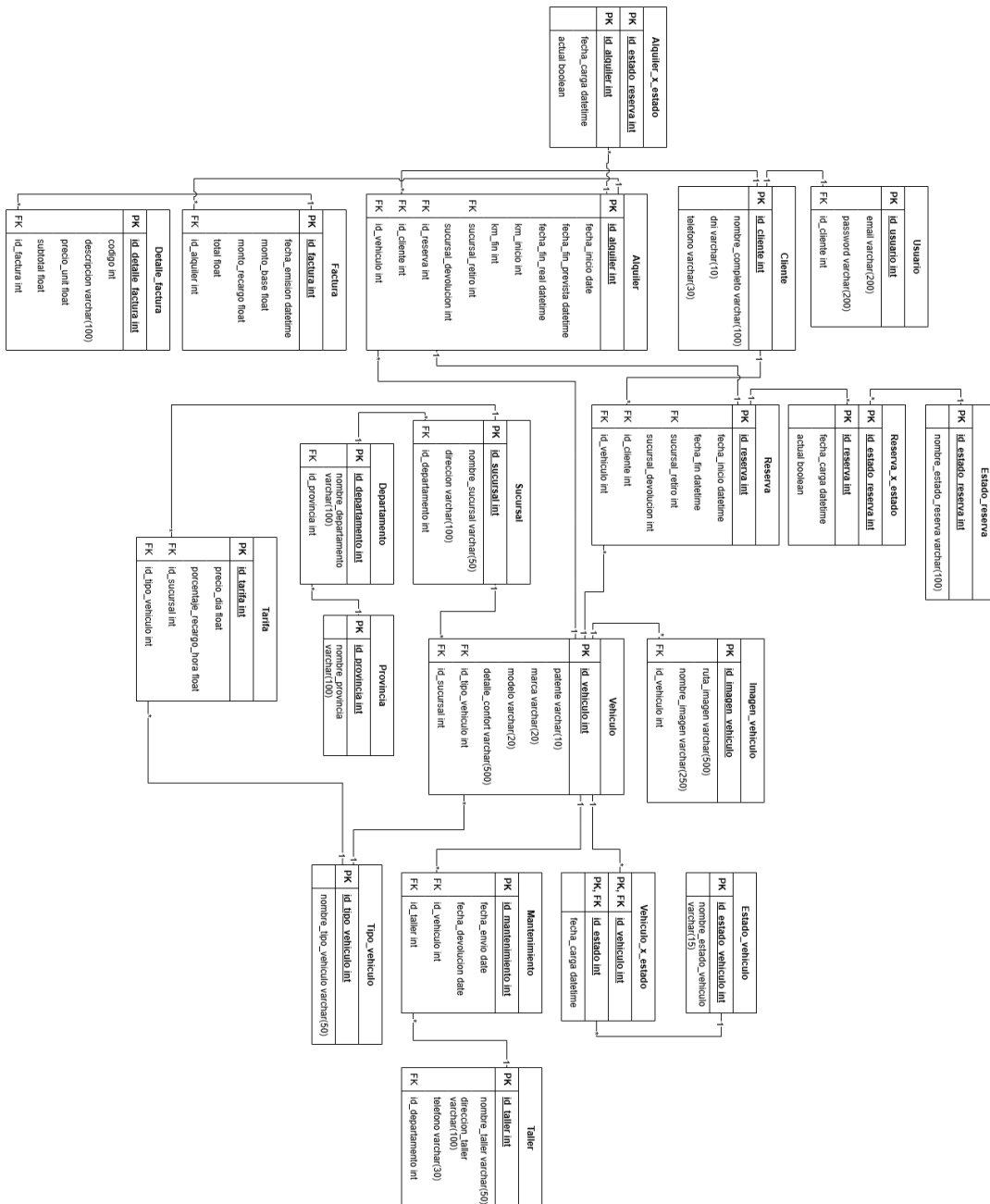
La reserva de un vehículo deberá validar la disponibilidad del mismo en las fechas solicitadas, evitando superposición con otros alquileres o reservas existentes.

El estado "en mantenimiento" del vehículo surge del envío del mismo a un taller mecánico, dicho envío debe ser registrado, y deberá incluirse la información de fecha envío y fecha de devolución.

Eta pa I

DER

Diagrama entidad-relación de la base de datos de la solución propuesta



Funcionalidades

1. Gestión de sucursales
 - Registrar, modificar y eliminar sucursales.
 - Consultar sucursales existentes.
 - Asociar vehículos a una sucursal.
2. Gestión de vehículos
 - Alta, baja y modificación de vehículos. => Tabla Vehiculo
 - Registrar atributos:
 - Marca, modelo, tipo (SUV, sedán, etc.).
 - Detalle de confort.
 - Estado (disponible, alquilado, mantenimiento).
 - Asignar vehículo a una sucursal.
 - Registrar entre 1 y 5 imágenes por vehículo.
 - Consultar disponibilidad de vehículos por fecha.
3. Gestión de clientes
 - Registro de clientes.
 - Gestión de usuario y contraseña (autenticación).
 - Consulta y actualización de datos del cliente.
4. Gestión de alquileres
 - Crear un alquiler indicando:
 - Cliente
 - Vehículo
 - Fecha inicio
 - Fecha devolución prevista
 - Registrar:
 - Kilómetros al inicio
 - Kilómetros al finalizar
 - Finalizar alquiler:
 - Calcular importe total
 - Cambiar estado del vehículo a disponible
 - Validar que el vehículo esté disponible.
5. Gestión de reservas
 - Permitir a clientes registrados:
 - Crear reservas online
 - Validaciones:
 - Verificar disponibilidad del vehículo en el rango de fechas
 - Evitar superposición con:
 - Otros alquileres
 - Otras reservas
 - Cancelar reservas.

- Convertir reserva en alquiler.
- 6. Gestión de tarifas
 - Definir valor diario del alquiler:
 - Por tipo de vehículo
 - Por sucursal
 - Definir porcentaje de recargo por hora excedida:
 - También por tipo y sucursal
- 7. Facturación
 - Generar factura al finalizar un alquiler:
 - Datos del cliente
 - Período alquilado
 - Precio base
 - Recargos por demora
 - Calcular recargo:
 - Si hay devolución fuera de término
 - Consultar facturas emitidas.
- 8. Gestión de mantenimiento
 - Registrar envío de vehículo a taller:
 - Fecha de envío
 - Fecha de devolución
 - Cambiar el estado del vehículo a “en mantenimiento”.
 - Al regresar:
 - Actualizar estado a disponible.
- 9. Control de estados del vehículo
 - Cambios automáticos de estado:
 - Disponible → Alquilado
 - Alquilado → Disponible
 - Disponible → Mantenimiento
 - Consultar estado actual.
- 10. Consultas y reportes
 - Listado de vehículos disponibles.
 - Historial de alquileres por cliente.
 - Historial de uso de un vehículo.
 - Reportes de ingresos.
 - Vehículos en mantenimiento.
- 11. Seguridad y acceso
 - Login de usuarios (clientes).
 - Posible gestión de roles (admin vs cliente).
 - Protección de operaciones críticas.

Matriz funcionalidad, tabla y justificación

Funcionalidad	Tabla	Justificación
Gestión de sucursales • Registrar, modificar y eliminar sucursales. • Consultar sucursales existentes. • Asociar vehículos a una sucursal.	Sucursal. Columna <i>id_sucursal</i> en la tabla vehículo.	La tabla sucursal posee los atributos para el almacenamiento de una sucursal. La tabla Vehiculo posee la Foreign Key <i>id_sucursal</i> que hace referencia a la tabla Sucursal
Gestión de vehículos • Alta, baja y modificación de vehículos. • Registrar atributos: ○ Marca, modelo, tipo (SUV, sedán, etc.). ○ Detalle de confort. ○ Estado (disponible, alquilado, mantenimiento). • Asignar vehículo a una sucursal. ○ Registrar entre 1 y 5 imágenes por vehículo. • Consultar disponibilidad de vehículos por fecha.	Vehiculo. Tipo_vehiculo. Estado_vehiculo. Imagen_vehiculo. Vehiculo_x_estado.	La tabla Vehículo posee los atributos para el almacenamiento de los vehículos. La tabla tipo_vehiculo posee los distintos tipos de vehículos. La tabla estado_vehiculo posee todos los posibles estados de un vehículo. La tabla imagen_vehiculo posee las imágenes de un vehículo. La tabla vehiculo_x_estado registra los distintos estados por lo que pasa un vehículo.

Funcionalidad	Tabla	Justificación
Gestión de clientes • Registro de clientes. • Gestión de usuario y contraseña (autenticación). • Consulta y actualización de datos del cliente.	Usuario Cliente	La tabla usuario posee los atributos para el almacenamiento de un usuario. La tabla cliente posee los atributos para el almacenamiento de un cliente.
Gestión de alquileres • Crear un alquiler indicando: ○ Cliente ○ Vehículo ○ Fecha inicio ○ Fecha devolución prevista • Registrar: ○ Kilómetros al inicio ○ Kilómetros al finalizar • Finalizar alquiler: ○ Calcular importe total ○ Cambiar estado del vehículo a disponible • Validar que el vehículo esté disponible.	Alquiler Tarifa Vehiculo_x_estado	La tabla alquiler posee los atributos para el almacenamiento de un alquiler de un vehículo. La tabla tarifa posee los atributos para el almacenamiento de las distintas tarifas de recargo por el tipo de vehículo. La tabla vehiculo_x_estado posee los atributos para el almacenamiento del historial de estados por lo que pasó un vehículo.
Gestión de reservas • Permitir a clientes registrados: ○ Crear reservas online • Validaciones: ○ Verificar disponibilidad del vehículo en el rango de fechas	Reserva	La tabla reserva posee los atributos para el almacenamiento de una reserva.

Funcionalidad	Tabla	Justificación
- Evitar superposición con: - Otros alquileres - Otras reservas - Cancelar reservas. - Convertir reserva en alquiler.		
Gestión de tarifas - Definir valor diario del alquiler: - Por tipo de vehículo - Por sucursal - Definir porcentaje de recargo por hora excedida: - También por tipo y sucursal	Tarifa	La tabla tarifa posee los atributos para el almacenamiento de las tarifas por tipo de vehículo.
Facturación - Generar factura al finalizar un alquiler: - Datos del cliente - Período alquilado - Precio base - Recargos por demora - Calcular recargo: - Si hay devolución fuera de término - Consultar facturas emitidas.	Factura Detalle_factura	La tabla factura posee los atributos para el almacenamiento de una factura. La tabla detalle_factura posee los atributos para el almacenamiento de los detalles de una factura.
Gestión de mantenimiento - Registrar envío de vehículo a taller: - Fecha de envío - Fecha de devolución - Cambiar el estado del vehículo a "en mantenimiento".	Mantenimiento Taller	La tabla taller posee los atributos para el almacenamiento de un taller.

Funcionalidad	Tabla	Justificación
- Al regresar: - Actualizar estado a disponible.		
Control de estados del vehículo - Cambios automáticos de estado: - Disponible → Alquilado - Alquilado → Disponible - Disponible → Mantenimiento - Consultar estado actual.	Estado_x_vehiculo	Uso de triggers
Consultas y reportes - Listado de vehículos disponibles. - Historial de alquileres por cliente. - Historial de uso de un vehículo. - Reportes de ingresos. - Vehículos en mantenimiento.	Vistas	Uso de Vistas
Seguridad y acceso - Login de usuarios (clientes). - Posible gestión de roles (admin vs cliente). - Protección de operaciones críticas.	Usuario Permiso Roles	Uso de transacciones.

DDL (MySQL)

```
CREATE TABLE `cliente` (  
    `id_cliente` int NOT NULL AUTO_INCREMENT,  
    `nombre_completo` varchar(100),  
    `dni` varchar(10),  
    `telefono` varchar(30),  
    primary key(`id_cliente`)  
);  
  
CREATE TABLE `usuario` (  
    `id_usuario` int NOT NULL AUTO_INCREMENT,  
    `email` varchar(200) UNIQUE NOT NULL,  
    `password` varchar(200) NOT NULL,  
    `id_cliente` int NOT NULL,  
    primary key(`id_usuario`),  
    CONSTRAINT `fk_usuario_x_cliente` FOREIGN KEY (`id_cliente`) REFERENCES  
`cliente` (`id_cliente`)  
);  
  
CREATE TABLE `provincia`(  
    `id_provincia` int not null AUTO_INCREMENT,  
    `nombre_provincia` varchar(100),  
    primary key(`id_provincia`)
```

```
);

CREATE TABLE `departamento` (
    `id_departamento` int not null AUTO_INCREMENT,
    `nombre_departamento` varchar(100),
    `id_provincia` int,
    primary key(`id_departamento`),
    CONSTRAINT `fk_departamento_x_provincia` FOREIGN KEY (`id_provincia`)
REFERENCES `provincia`(`id_provincia`)
);

CREATE TABLE `sucursal` (
    `id_sucursal` int not NULL AUTO_INCREMENT,
    `nombre_sucursal` varchar(50) NOT NULL,
    `direccion_sucursal` varchar(100),
    `id_departamento` int,
    PRIMARY KEY(`id_sucursal`),
    CONSTRAINT `fk_sucursal_x_departamento` FOREIGN KEY (`id_departamento`)
REFERENCES `departamento`(`id_departamento`)
);

CREATE TABLE `tipo_vehiculo` (
    `id_tipo_vehiculo` INT NOT NULL AUTO_INCREMENT,
```

```
`nombre_tipo_vehiculo` varchar(50) NOT NULL,  
  
PRIMARY KEY(`id_tipo_vehiculo`)  
);  
  
CREATE TABLE `taller` (  
  
    `id_taller` int NOT NULL AUTO_INCREMENT,  
  
    `nombre_taller` varchar(50) NOT NULL,  
  
    `direccion_taller` varchar(100),  
  
    `telefono_taller` varchar(30),  
  
    `id_departamento` int,  
  
    primary key(`id_taller`),  
  
    CONSTRAINT `fk_taller_x_departamento` FOREIGN KEY (`id_departamento`)  
REFERENCES `departamento`(`id_departamento`)  
);  
  
CREATE TABLE `estado_vehiculo` (  
  
    `id_estado_vehiculo` int not null AUTO_INCREMENT,  
  
    `nombre_estado_vehiculo` varchar(15),  
  
    primary key(`id_estado_vehiculo`)  
);  
  
CREATE TABLE `vehiculo`(  
  
    `id_vehiculo` int not null AUTO_INCREMENT,  
  
    `patente` varchar(10),
```

```
`marca` varchar(20),  
  
`modelo` varchar(20),  
  
`detalle_comfort` varchar(500),  
  
`id_tipo_vehiculo` int,  
  
`id_sucursal` int,  
  
primary key(`id_vehiculo`),  
  
CONSTRAINT `fk_vehiculo_x_tipo_vehiculo` FOREIGN KEY  
(`id_tipo_vehiculo`) REFERENCES `tipo_vehiculo`(`id_tipo_vehiculo`),  
  
CONSTRAINT `fk_vehiculo_x_sucursal` FOREIGN KEY (`id_sucursal`)  
REFERENCES `sucursal`(`id_sucursal`)  
);  
  
CREATE TABLE `imagen_vehiculo` (  
  
    `id_imagen_vehiculo` int not null AUTO_INCREMENT,  
  
    `ruta_imagen` varchar(500),  
  
    `nombre_imagen` varchar(250),  
  
    `id_vehiculo` int,  
  
    primary key(`id_imagen_vehiculo`),  
  
    CONSTRAINT `fk_imgveh_x_veh` FOREIGN KEY (`id_vehiculo`) REFERENCES  
`vehiculo`(`id_vehiculo`)  
);  
  
CREATE TABLE `vehiculo_x_estado` (  
  
    `id_veh_x_est` int not null AUTO_INCREMENT,
```

```
`id_vehiculo` int not null,  
`id_estado_vehiculo` int not null,  
primary key(`id_veh_x_est`)  
);  
  
CREATE TABLE `mantenimiento` (  
    `id_mantenimiento` int not null AUTO_INCREMENT,  
    `fecha_envio` date,  
    `fecha_devolucion` date,  
    `id_vehiculo` int,  
    `id_taller` int,  
    primary key(`id_mantenimiento`),  
    CONSTRAINT `fk_mant_x_veh` FOREIGN KEY (`id_vehiculo`) REFERENCES  
`vehiculo` (`id_vehiculo`),  
    CONSTRAINT `fk_mant_x_taller` FOREIGN KEY (`id_taller`) REFERENCES  
`taller` (`id_taller`)  
);  
  
CREATE TABLE `tarifa` (  
    `id_tarifa` int not null AUTO_INCREMENT,  
    `precio_dia` decimal(10,2),  
    `porcentaje_recargo_hora` decimal(10,2),  
    `id_sucursal` int,  
    `id_tipo_vehiculo` int,
```



```
PRIMARY KEY(`id_tarifa`),

    CONSTRAINT `fk_tarifa_x_tipo_vehiculo` FOREIGN KEY (`id_tipo_vehiculo`)
REFERENCES `tipo_vehiculo`(`id_tipo_vehiculo`),

    CONSTRAINT `fk_tarifa_x_sucursal` FOREIGN KEY (`id_sucursal`) REFERENCES
`sucursal`(`id_sucursal`)
);

CREATE TABLE `reserva` (

    `id_reserva` int not null AUTO_INCREMENT,

    `fecha_inicio` datetime not null,

    `fecha_fin` datetime not null,

    `sucursal_retiro` int not null,

    `sucursal_devolucion` int,

    `id_cliente` int not null,

    `id_vehiculo` int not null,

    primary key(`id_reserva`),

    CONSTRAINT `fk_reserv_x_cliente` FOREIGN KEY (`id_cliente`) REFERENCES
`cliente`(`id_cliente`),

    CONSTRAINT `fk_reserv_x_veh` FOREIGN KEY (`id_vehiculo`) REFERENCES
`vehiculo`(`id_vehiculo`),

    CONSTRAINT `fk_reserv_x_sucursal` FOREIGN KEY (`sucursal_retiro`)
REFERENCES `sucursal`(`id_sucursal`)
);

CREATE TABLE `estado_reserva` (
```

```
`id_estado_reserva` int not null AUTO_INCREMENT,  
  
`nombre_estado_reserva` varchar(15),  
  
primary key(`id_estado_reserva`)  
);  
  
CREATE TABLE `reserva_x_estado` (  
  
    `id_reserva` int,  
  
    `id_estado_reserva` int,  
  
    primary key(`id_reserva`, `id_estado_reserva`),  
  
    CONSTRAINT `fk_rxe_x_reserv` FOREIGN KEY (`id_reserva`) REFERENCES  
`reserva`(`id_reserva`),  
  
    CONSTRAINT `fk_rxe_x_estado_reserv` FOREIGN KEY (`id_estado_reserva`)  
REFERENCES `estado_reserva`(`id_estado_reserva`)  
);  
  
CREATE TABLE `alquiler`(  
  
    `id_alquiler` int not null AUTO_INCREMENT,  
  
    `fecha_inicio` datetime not null,  
  
    `fecha_fin_prevista` datetime not null,  
  
    `fecha_fin_real` datetime,  
  
    `km_inicio` int not null,  
  
    `km_fin` int,  
  
    `sucursal_retiro` int not null,  
  
    `sucursal_devolucion` int,
```

```
`id_reserva` int,  
`id_cliente` int,  
`id_vehiculo` int,  
PRIMARY KEY(`id_alquiler`),  
CONSTRAINT `fk_alquiler_x_cliente` FOREIGN KEY (`id_cliente`) REFERENCES  
`cliente`(`id_cliente`),  
CONSTRAINT `fk_alquiler_x_veh` FOREIGN KEY (`id_vehiculo`) REFERENCES  
`vehiculo`(`id_vehiculo`),  
CONSTRAINT `fk_alquiler_x_reserv` FOREIGN KEY (`id_reserva`) REFERENCES  
`reserva`(`id_reserva`),  
CONSTRAINT `fk_alquiler_x_sucursal` FOREIGN KEY (`sucursal_retiro`) REFERENCES  
`sucursal`(`id_sucursal`)  
);  
  
CREATE TABLE `alquiler_x_estado` (  
    `id_alquiler` int not null,  
    `id_estado_reserva` int not null,  
    primary key(`id_alquiler`, `id_estado_reserva`),  
    CONSTRAINT `fk_axe_x_alquiler` FOREIGN KEY (`id_alquiler`) REFERENCES  
    `alquiler`(`id_alquiler`),  
    CONSTRAINT `fk_axe_x_estado_reserv` FOREIGN KEY (`id_estado_reserva`) REFERENCES  
    `estado_reserva`(`id_estado_reserva`)  
);  
  
CREATE TABLE `factura` (  

```

```
`id_factura` int not null AUTO_INCREMENT,  
  
`fecha_emision` datetime not null,  
  
`monto_base` decimal(10,2),  
  
`monto_recargo` decimal(10,2),  
  
`total` decimal(10,2),  
  
`id_alquiler` int not null,  
  
PRIMARY KEY(`id_factura`),  
  
CONSTRAINT `fk_factura_x_alquiler` FOREIGN KEY (`id_alquiler`)  
REFERENCES `alquiler`(`id_alquiler`)  
);  
  
CREATE TABLE `detalle_factura` (  
  
    `id_detalle_factura` int not null AUTO_INCREMENT,  
  
    `codigo` int,  
  
    `descripcion` varchar(100),  
  
    `precio_unitario` decimal(10,2),  
  
    `subtotal` decimal(10,2),  
  
    `id_factura` int,  
  
    PRIMARY KEY(`id_detalle_factura`),  
  
    CONSTRAINT `fk_det_factura_x_factura` FOREIGN KEY (`id_factura`)  
REFERENCES `factura`(`id_factura`)  
);
```

Requerimientos Técnicos – Etapas de Implementación

- Implementación de mecanismos de auditoría mediante triggers, registrando las operaciones realizadas sobre las tablas principales del sistema.

Los logs deberán almacenar:

- Usuario que realizó la operación,
- Fecha y hora,
- Tipo de operación (INSERT, UPDATE o DELETE).

Asimismo:

- Para operaciones INSERT deberán almacenarse los valores nuevos.
- Para operaciones DELETE deberán almacenarse los valores anteriores.
- Para operaciones UPDATE deberán almacenarse tanto los valores anteriores como los nuevos.

La auditoría podrá implementarse mediante:

- Una tabla de log general para todo el sistema o tablas de log específicas por cada entidad auditada.

Realizar una interfaz desde el sistema a desarrollar que permita la consulta de estos logs.

- Todos los procedimientos almacenados deberán implementar manejo de excepciones, contemplando:
 - Control de errores.
 - Mensajes de retorno.
 - aplicación de COMMIT y ROLLBACK en aquellos procesos que involucren transacciones.
- Desarrollo de operaciones CRUD (Create, Read, Update, Delete) para las entidades principales del sistema.

Dichas operaciones deberán implementarse mediante programación en base de datos y trabajar obligatoriamente con datos previamente cargados en el sistema.

- Toda la lógica de inserción, actualización y eliminación de información deberá desarrollarse mediante PL/SQL y/o procedimientos almacenados equivalentes según el SGBD seleccionado.

Los procedimientos deberán retornar información indicando:

- Éxito de la operación.
- Errores producidos.
- Validaciones de negocio no cumplidas.



Implementación y utilización de parámetros:

- IN.
- OUT.
- IN OUT.

En los distintos procedimientos almacenados desarrollados.

- El sistema deberá permitir procesar alquileres:
 - Con reserva previa.
 - Directamente sin reserva.

Ambas modalidades deberán ser contempladas y correctamente validadas.

- Desarrollo de un procedimiento almacenado encargado de registrar reservas, validando previamente:
 - Disponibilidad del vehículo.
 - Superposición de fechas.
 - Demás restricciones necesarias.

El mismo criterio deberá aplicarse al momento de registrar alquileres.

Se valorará especialmente:

- Modularización.
- Reutilización de código.
- Separación de responsabilidades.

Implementación de funcionalidad de cancelación/baja de reservas.

La solución deberá contemplar las validaciones necesarias según el estado de la reserva.

- Desarrollo de jobs/tareas programadas que se ejecuten automáticamente en horarios definidos, con el objetivo de detectar vehículos cuya fecha prevista de devolución haya expirado y aún no hayan sido entregados.

La información detectada deberá almacenarse en estructuras específicas de auditoría/reportes históricos diseñadas para tal fin.

- Implementación del proceso de finalización de alquiler, el cual deberá:
 - Registrar la devolución del vehículo.
 - Actualizar automáticamente el estado del mismo.
 - Calcular recargos correspondientes,
 - Generar la factura asociada al alquiler.

Parte de esta lógica deberá resolverse mediante triggers y/o programación en base de datos.

Etapa II

Base de datos: PostgreSQL.

Aplicación: Laravel PHP.

Tabla de control de la base de datos

		Procedimiento			Auditoría		
Tabla	Tabla Log	Alta	Baja	Modificación	Secuencia	Trigger	Función Trigger
Alquiler	SI	SI	SI	SI	SI	SI	SI
Alquiler_x_estado	SI	-	-	-	SI	SI	SI
Cliente	-	-	-	-	-	-	-
Departamento	-	-	-	-	-	-	-
Detalle_factura	SI	-	-	-	SI	SI	SI
Estado_alquiler	-	-	-	-	-	-	-
Estado_reserva	-	-	-	-	-	-	-
Estado_vehiculo	-	-	-	-	-	-	-
Factura	SI	SI	-	-	SI	SI	SI
Imagen_vehiculo	-	-	-	-	-	-	-
Mantenimiento	SI	SI	SI	SI	SI	SI	SI
Provincia	-	-	-	-	-	-	-
Reserva	SI	SI	SI	SI	SI	SI	SI
Reserva_x_estado	SI	-	-	-	SI	SI	SI
Sucursal	SI	SI	SI	SI	SI	SI	SI
Taller	SI	SI	SI	SI	SI	SI	SI
Tarifa	SI	SI	SI	SI	SI	SI	SI
Tipo_vehiculo	SI	SI	SI	SI	SI	SI	SI

		Procedimiento			Auditoría		
Tabla	Tabla Log	Alta	Baja	Modificación	Secuencia	Trigger	Función Trigger
Usuario	-	-	-	-	-	-	-
Vehículo	SI	SI	SI	SI	SI	SI	SI
Vehiculo_x_estado	SI	-	-	-	SI	SI	SI

Esquema

Archivo: [database.sql](#)

Secuencias

Archivo: [database-secuencias.sql](#)

Insert Iniciales

Archivo: [database-insert-ini.sql](#)

Funciones

Archivo: [database-funciones.sql](#)

Índices

Archivo: [database-indices.sql](#)

Procedimientos

Archivo: [database-procedimientos.sql](#)

Esquema Logs

Archivo: [database-logs.sql](#)

Triggers

Archivo: [database-triggers.sql](#)

Jobs

Archivo: [database-jobs.sql](#)

Observaciones:

- PostgreSQL implementa control transaccional implícito en bloques PL/pgSQL con manejo de excepciones, garantizando atomicidad mediante subtransacciones automáticas.
- Para implementar Jobs en PostgreSQL, normalmente se implementa con una extensión como **pg_cron**.

Procedimientos, funciones y explicación:

Función o Procedimiento	Tabla	Explicación
sp_alta_vehiculo	vehículo	Crea un vehículo nuevo.
sp_alta_imagen_vehiculo	imagen_vehiculo	Crea la imagen de un vehiculo.
sp_modificar_vehiculo	vehículo	Modifica los datos de un vehículo existente.
sp_baja_vehiculo	vehículo	Realiza baja lógica o eliminación del vehículo.
sp_cambiar_sucursal_vehiculo	vehículo	Permite mover un vehículo a otra sucursal.
sp_alta_reserva	reserva	Genera una reserva nueva.
sp_modificar_reserva	reserva	Modifica los datos de una reserva existente.
sp_baja_reserva	reserva	Cancela o elimina una reserva.
sp_alta_alquiler	alquiler	Crea un alquiler.
sp_modificar_alquiler	alquiler	Actualiza información del alquiler.
sp_baja_alquiler	alquiler	Cancela o elimina un alquiler.
sp_finalizar_alquiler	alquiler Factura	Finaliza un alquiler y genera la factura

Función o Procedimiento	Tabla	Explicación
	Detalle_factura	
sp_generar_alquiler_desde_reserva	alquiler	Migra los datos de la tabla reserva a la tabla alquiler
sp_alta_taller	taller	Crea un taller nuevo.
sp_modificar_taller	taller	Actualiza datos del taller.
sp_baja_taller	taller	Elimina o desactiva un taller.
sp_alta_mantenimiento	mantenimiento	Registra un mantenimiento.
sp_modificar_mantenimiento	mantenimiento	Actualiza datos del mantenimiento.
sp_baja_mantenimiento	mantenimiento	Elimina o cancela mantenimiento.
sp_finalizar_mantenimiento	mantenimiento	Carga fecha devolución del taller.
sp_alta_tarifa	tarifa	Crea tarifas.
sp_modificar_tarifa	tarifa	Actualiza precios y recargos.
sp_baja_tarifa	tarifa	Elimina o desactiva tarifas.
fn_vehiculo_disponible	reserva, alquiler	Verifica si un vehículo: - Está disponible. - No tiene reservas superpuestas. - No tiene alquileres superpuestos.
fn_alquileres_vencidos		Devuelve alquileres: - Vencidos. - No entregados. - Aún activos.
fn_calcular_monto_tarifa		Retorna el monto total, desde la fecha de retiro, la fecha de

Función o Procedimiento	Tabla	Explicación
		devolución y la tarifa.
jb_generar_facturas_masivas()		Procedimiento masivo: Recorrer todos los alquileres finalizados que aún no poseen factura.
jb_cancelar_reservas_vencidas		Actualización masiva de estados de reservas vencidas: Una reserva ya pasó su fecha de inicio y/o nunca se convirtió en alquiler, entonces se marca automáticamente como vencida.



Roles:

- **CLIENTE**
 - Crear reservas
 - Consultar disponibilidad
 - Ver sus alquileres y facturas
- **EMPLEADO**
 - Gestionar reservas
 - Crear alquileres
 - Registrar devoluciones
- **GERENTE**
 - Administrar vehículos de su sucursal
 - Ver reportes locales
 - Gestionar mantenimientos



Front End: Aplicación Laravel

Vista Principal

Inicio

Sucursal

Vehículo

Reserva

Reservados

Log

?

AutoGest

Sucursal de retiro

Fecha de retiro

Fecha de devolución

Buscar vehículos disponibles

Seleccione una sucursal

dd/mm/aaaa --:--

dd/mm/aaaa --:--

☒ Devolución en la misma oficina.

Copyright © AutoGest 2026

Inicio

Sucursal

Vehículo

Reserva

Reservados

Log

?

AutoGest

Sucursal de retiro

Fecha de retiro

Fecha de devolución

Buscar vehículos disponibles

Sucursal 1

15/06/2026 12:35

19/06/2026 12:35

☒ Devolución en la misma oficina.

Fiat Cronos2

un autazo

Sedan

Kilometraje ilimitado

Seguro colisión (CDW)

Seguro robo

\$ 175.002,50

Reservar

Sucursal Sucursal 1, Capital

ABM Sucursal

Sucursales

[+ Crear](#)

Mostrar registros por página

Buscar:

Cod.	Nombre	Dirección	Departamento	Acciones
2	Sucursal 1	direccion 12	Capital	✎ ✖
3	sucursal 3	direccion 2113	San Ignacio	✎ ✖

Mostrando página 1 de 1

Ant. **1** Sig.

Nueva Sucursal

Nombre de la sucursal

Dirección

Departamento

Seleccione un departamento ▼

[Guardar](#)
[← Volver](#)

ABM Vehiculo

Vehiculos

[+ Crear](#)

Mostrar registros por página

Buscar:

Cod.	Patente	Marca	Modelo	Detalle	Estado	Tipo	Sucursal	Acciones
17	ACB123ZZ	Fiat	600	HYBRID	EN ESPERA	CUPE	Sucursal 1	✎ ✖
5	AAA212LO	Fiat	Cronitos	un autazo	DISPONIBLE	Sedan	Sucursal 1	✎ ✖
7	AAA212TR	Fiat	Cronitos2	un autaz2o	EN ESPERA	Sedan	Sucursal 1	✎ ✖
1	AAA212SS	Fiat	Cronos	un autazo	BAJA	Sedan	Sucursal 1	✎ ✖
6	AAA212HS	Fiat	Cronos2	un autazo	EN ESPERA	Sedan	Sucursal 1	✎ ✖
18	AAA123WW	Fiat	se	sadaw	BAJA	CUPE	Sucursal 1	✎ ✖

Mostrando página 1 de 1

Ant. **1** Sig.

Nuevo Vehículo

Patente

Marca

Modelo

Detalle de confort

Tipo de vehículo

Seleccione un tipo

Sucursal

Seleccione una sucursal

Imágenes del vehículo

Elegir archivos

Ningún archivo seleccionado

Guardar

Volver

ANEXO

DDL para PostgreSQL

```
CREATE TABLE cliente (  
    id_cliente SERIAL PRIMARY KEY,  
    nombre_completo VARCHAR(100),  
    dni VARCHAR(10),  
    telefono VARCHAR(30)  
);  
  
CREATE TABLE usuario (  
    id_usuario SERIAL PRIMARY KEY,  
    email VARCHAR(200) UNIQUE NOT NULL,  
    password VARCHAR(200) NOT NULL,  
    id_cliente INT NOT NULL,  
    CONSTRAINT fk_usuario_x_cliente  
        FOREIGN KEY (id_cliente)  
        REFERENCES cliente(id_cliente)  
);  
  
CREATE TABLE provincia (  
    id_provincia SERIAL PRIMARY KEY,  
    nombre_provincia VARCHAR(100)  
);
```



```
CREATE TABLE departamento (  
    id_departamento SERIAL PRIMARY KEY,  
    nombre_departamento VARCHAR(100),  
    id_provincia INT,  
    CONSTRAINT fk_departamento_x_provincia  
        FOREIGN KEY (id_provincia)  
        REFERENCES provincia(id_provincia)  
);  
  
CREATE TABLE sucursal (  
    id_sucursal SERIAL PRIMARY KEY,  
    nombre_sucursal VARCHAR(50) NOT NULL,  
    direccion_sucursal VARCHAR(100),  
    id_departamento INT,  
    CONSTRAINT fk_sucursal_x_departamento  
        FOREIGN KEY (id_departamento)  
        REFERENCES departamento(id_departamento)  
);  
  
CREATE TABLE tipo_vehiculo (  
    id_tipo_vehiculo SERIAL PRIMARY KEY,  
    nombre_tipo_vehiculo VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE taller (  
    id_taller SERIAL PRIMARY KEY,  
    nombre_taller VARCHAR(50) NOT NULL,  
    direccion_taller VARCHAR(100),  
    telefono_taller VARCHAR(30),  
    id_departamento INT,  
    CONSTRAINT fk_taller_x_departamento  
        FOREIGN KEY (id_departamento)  
        REFERENCES departamento(id_departamento)  
);
```

```
CREATE TABLE estado_vehiculo (  
    id_estado_vehiculo SERIAL PRIMARY KEY,  
    nombre_estado_vehiculo VARCHAR(15)  
);
```

```
CREATE TABLE vehiculo (  
    id_vehiculo SERIAL PRIMARY KEY,  
    patente VARCHAR(10),  
    marca VARCHAR(20),  
    modelo VARCHAR(20),  
    detalle_confort VARCHAR(500),
```

```
id_tipo_vehiculo INT,  
  
id_sucursal INT,  
  
CONSTRAINT fk_vehiculo_x_tipo_vehiculo  
    FOREIGN KEY (id_tipo_vehiculo)  
    REFERENCES tipo_vehiculo(id_tipo_vehiculo),  
  
CONSTRAINT fk_vehiculo_x_sucursal  
    FOREIGN KEY (id_sucursal)  
    REFERENCES sucursal(id_sucursal)  
);  
  
CREATE TABLE imagen_vehiculo (  
    id_imagen_vehiculo SERIAL PRIMARY KEY,  
    ruta_imagen VARCHAR(500),  
    nombre_imagen VARCHAR(250),  
    id_vehiculo INT,  
  
    CONSTRAINT fk_imgveh_x_veh  
        FOREIGN KEY (id_vehiculo)  
        REFERENCES vehiculo(id_vehiculo)  
);
```

```
CREATE TABLE vehiculo_x_estado (  
    id_veh_x_est SERIAL PRIMARY KEY,  
    id_vehiculo INT NOT NULL,  
    id_estado_vehiculo INT NOT NULL,  
    fecha_estado TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    CONSTRAINT fk_vxe_x_veh  
        FOREIGN KEY (id_vehiculo)  
        REFERENCES vehiculo(id_vehiculo),  
  
    CONSTRAINT fk_vxe_x_estado  
        FOREIGN KEY (id_estado_vehiculo)  
        REFERENCES estado_vehiculo(id_estado_vehiculo)  
);
```

```
CREATE TABLE mantenimiento (  
    id_mantenimiento SERIAL PRIMARY KEY,  
    fecha_envio DATE,  
    fecha_devolucion DATE,  
    id_vehiculo INT,  
    id_taller INT,  
  
    CONSTRAINT fk_mant_x_veh
```

```
FOREIGN KEY (id_vehiculo)

REFERENCES vehiculo(id_vehiculo),

CONSTRAINT fk_mant_x_taller

FOREIGN KEY (id_taller)

REFERENCES taller(id_taller)

);

CREATE TABLE tarifa (

    id_tarifa SERIAL PRIMARY KEY,

    precio_dia NUMERIC(10,2),

    porcentaje_recargo_hora NUMERIC(10,2),

    id_sucursal INT,

    id_tipo_vehiculo INT,

    CONSTRAINT fk_tarifa_x_tipo_vehiculo

        FOREIGN KEY (id_tipo_vehiculo)

        REFERENCES tipo_vehiculo(id_tipo_vehiculo),

    CONSTRAINT fk_tarifa_x_sucursal

        FOREIGN KEY (id_sucursal)

        REFERENCES sucursal(id_sucursal)

);
```

```
CREATE TABLE reserva (  
    id_reserva SERIAL PRIMARY KEY,  
    fecha_inicio TIMESTAMP NOT NULL,  
    fecha_fin TIMESTAMP NOT NULL,  
    sucursal_retiro INT NOT NULL,  
    sucursal_devolucion INT,  
    id_cliente INT NOT NULL,  
    id_vehiculo INT NOT NULL,  
  
    CONSTRAINT fk_reserv_x_cliente  
        FOREIGN KEY (id_cliente)  
        REFERENCES cliente(id_cliente),  
  
    CONSTRAINT fk_reserv_x_veh  
        FOREIGN KEY (id_vehiculo)  
        REFERENCES vehiculo(id_vehiculo),  
  
    CONSTRAINT fk_reserv_x_sucursal_retiro  
        FOREIGN KEY (sucursal_retiro)  
        REFERENCES sucursal(id_sucursal),  
  
    CONSTRAINT fk_reserv_x_sucursal_dev
```

```
FOREIGN KEY (sucursal_devolucion)
REFERENCES sucursal(id_sucursal)
);

CREATE TABLE estado_reserva (
    id_estado_reserva SERIAL PRIMARY KEY,
    nombre_estado_reserva VARCHAR(15)
);

CREATE TABLE reserva_x_estado (
    id_reserva INT,
    id_estado_reserva INT,
    fecha_estado TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_reserva, id_estado_reserva),

    CONSTRAINT fk_rxe_x_reserv
        FOREIGN KEY (id_reserva)
        REFERENCES reserva(id_reserva),

    CONSTRAINT fk_rxe_x_estado_reserv
        FOREIGN KEY (id_estado_reserva)
        REFERENCES estado_reserva(id_estado_reserva)
);
```

```
CREATE TABLE alquiler (
    id_alquiler SERIAL PRIMARY KEY,
    fecha_inicio TIMESTAMP NOT NULL,
    fecha_fin_prevista TIMESTAMP NOT NULL,
    fecha_fin_real TIMESTAMP,
    km_inicio INT NOT NULL,
    km_fin INT,
    sucursal_retiro INT NOT NULL,
    sucursal_devolucion INT,
    id_reserva INT,
    id_cliente INT,
    id_vehiculo INT,

    CONSTRAINT fk_alquiler_x_cliente
        FOREIGN KEY (id_cliente)
        REFERENCES cliente(id_cliente),

    CONSTRAINT fk_alquiler_x_veh
        FOREIGN KEY (id_vehiculo)
        REFERENCES vehiculo(id_vehiculo),

    CONSTRAINT fk_alquiler_x_reserv
```



```
FOREIGN KEY (id_reserva)

REFERENCES reserva(id_reserva),

CONSTRAINT fk_alquiler_x_sucursal_ret

FOREIGN KEY (sucursal_retiro)

REFERENCES sucursal(id_sucursal),

CONSTRAINT fk_alquiler_x_sucursal_dev

FOREIGN KEY (sucursal_devolucion)

REFERENCES sucursal(id_sucursal)

);

CREATE TABLE estado_alquiler (

    id_estado_alquiler SERIAL PRIMARY KEY,

    nombre_estado_alquiler VARCHAR(15) UNIQUE

);

CREATE TABLE alquiler_x_estado (

    id_alquiler INT NOT NULL,

    id_estado_alquiler INT NOT NULL,

    fecha_estado TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    PRIMARY KEY (id_alquiler, id_estado_alquiler),
```

```
CONSTRAINT fk_axe_x_alquiler
    FOREIGN KEY (id_alquiler)
    REFERENCES alquiler(id_alquiler),

CONSTRAINT fk_axe_x_estado_reserv
    FOREIGN KEY (id_estado_alquiler)
    REFERENCES estado_alquiler(id_estado_alquiler)
);

CREATE TABLE factura (
    id_factura SERIAL PRIMARY KEY,
    fecha_emision TIMESTAMP NOT NULL,
    monto_base NUMERIC(10,2),
    monto_recargo NUMERIC(10,2),
    total NUMERIC(10,2),
    id_alquiler INT NOT NULL,

    CONSTRAINT fk_factura_x_alquiler
        FOREIGN KEY (id_alquiler)
        REFERENCES alquiler(id_alquiler)
);

CREATE TABLE detalle_factura (
    id_detalle_factura SERIAL PRIMARY KEY,
```

```
codigo INT,  
descripcion VARCHAR(100),  
precio_unitario NUMERIC(10,2),  
subtotal NUMERIC(10,2),  
id_factura INT,  
  
CONSTRAINT fk_det_factura_x_factura  
    FOREIGN KEY (id_factura)  
    REFERENCES factura(id_factura)  
);
```

Funciones

```
CREATE OR REPLACE FUNCTION fn_obtener_mensaje(  
    p_codigo_mensaje INTEGER  
)  
RETURNS VARCHAR  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_mensaje VARCHAR;  
BEGIN  
  
    SELECT  
        mensaje INTO v_mensaje  
    FROM mensaje  
    WHERE codigo_mensaje= p_codigo_mensaje;  
  
    RETURN v_mensaje;  
  
EXCEPTION  
    WHEN OTHERS THEN  
        RETURN 'Error desconocido';  
END;  
$$;
```

```
CREATE OR REPLACE FUNCTION fn_vehiculo_disponible(  
    p_id_vehiculo INT,  
    p_fecha_inicio TIMESTAMP,  
    p_fecha_fin TIMESTAMP,  
    p_id_reserva_excluir INT DEFAULT NULL  
)  
  
RETURNS BOOLEAN  
  
LANGUAGE plpgsql  
  
AS $$  
  
DECLARE  
    v_estado_actual INT;  
  
BEGIN  
  
    -- Validar existencia vehículo  
  
    IF NOT EXISTS (  
        SELECT 1  
        FROM vehiculo  
        WHERE id_vehiculo = p_id_vehiculo  
    ) THEN  
        RETURN FALSE;  
    END IF;  
  
    -- Obtener último estado
```

```
SELECT id_estado_vehiculo
INTO v_estado_actual
FROM vehiculo_x_estado
WHERE id_vehiculo = p_id_vehiculo
ORDER BY fecha_estado DESC
LIMIT 1;

-- Disponible o reservado
IF v_estado_actual NOT IN (1,2) THEN
    RETURN FALSE;
END IF;

-- Reservas superpuestas
IF EXISTS (
    SELECT 1
    FROM reserva
    WHERE id_vehiculo = p_id_vehiculo
    AND (
        p_id_reserva_excluir IS NULL
        OR id_reserva <> p_id_reserva_excluir
    )
    AND (
        p_fecha_inicio BETWEEN fecha_inicio AND fecha_fin
```

```
OR p_fecha_fin BETWEEN fecha_inicio AND fecha_fin

OR (

    p_fecha_inicio <= fecha_inicio

    AND p_fecha_fin >= fecha_fin

)

)

) THEN

    RETURN FALSE;

END IF;


-- Alquileros superpuestos

IF EXISTS (

    SELECT 1

    FROM alquiler

    WHERE id_vehiculo = p_id_vehiculo

    AND (

        p_fecha_inicio BETWEEN fecha_inicio AND fecha_fin_prevista

        OR p_fecha_fin BETWEEN fecha_inicio AND fecha_fin_prevista

        OR (

            p_fecha_inicio <= fecha_inicio

            AND p_fecha_fin >= fecha_fin_prevista

        )

    )

)
```

```
) THEN

    RETURN FALSE;

END IF;

RETURN TRUE;

END;

$$;

CREATE OR REPLACE FUNCTION fn_vehiculo_disponible_modificacion(
    p_id_reserva INT,
    p_id_vehiculo INT,
    p_fecha_inicio TIMESTAMP,
    p_fecha_fin TIMESTAMP
)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_actual INT;
BEGIN

    -- Validar vehículo
```



```
IF NOT EXISTS (  
    SELECT 1  
    FROM vehiculo  
    WHERE id_vehiculo = p_id_vehiculo  
) THEN  
    RETURN FALSE;  
END IF;  
  
-- Último estado  
SELECT ve.id_estado_vehiculo  
INTO v_estado_actual  
FROM vehiculo_x_estado ve  
WHERE ve.id_vehiculo = p_id_vehiculo  
ORDER BY ve.fecha_estado DESC  
LIMIT 1;  
  
-- Debe estar disponible o reservado  
IF v_estado_actual NOT IN (1, 2) THEN  
    RETURN FALSE;  
END IF;  
  
-- Reservas superpuestas  
IF EXISTS (  
    SELECT 1  
    FROM vehiculo_x_estado ve  
    WHERE ve.id_vehiculo = p_id_vehiculo  
    AND ve.fecha_estado < p_fecha_estado  
    AND ve.id_estado_vehiculo = p_id_estado_vehiculo  
    AND ve.id_estado_vehiculo IN (1, 2)
```

```
SELECT 1

FROM reserva r

WHERE r.id_vehiculo = p_id_vehiculo

AND r.id_reserva <> p_id_reserva

AND (

    p_fecha_inicio BETWEEN r.fecha_inicio AND r.fecha_fin

    OR p_fecha_fin BETWEEN r.fecha_inicio AND r.fecha_fin

    OR (

        p_fecha_inicio <= r.fecha_inicio

        AND p_fecha_fin >= r.fecha_fin

    )

)

) THEN

    RETURN FALSE;

END IF;

-- Alquileres superpuestos

IF EXISTS (

    SELECT 1

    FROM alquiler a

    WHERE a.id_vehiculo = p_id_vehiculo

    AND (

        p_fecha_inicio BETWEEN a.fecha_inicio AND a.fecha_fin_prevista
```

```
OR p_fecha_fin BETWEEN a.fecha_inicio AND a.fecha_fin_prevista  
  
OR (  
  
    p_fecha_inicio <= a.fecha_inicio  
  
    AND p_fecha_fin >= a.fecha_fin_prevista  
  
    )  
  
    )  
  
    ) THEN  
  
        RETURN FALSE;  
  
    END IF;  
  
    RETURN TRUE;  
  
END;  
$$;
```

Procedimientos

Sucursal

```
CREATE OR REPLACE PROCEDURE sp_alta_sucursal(  
    IN p_nombre_sucursal VARCHAR,  
    IN p_direccion_sucursal VARCHAR,  
    IN p_id_departamento INT,  
    OUT p_codigo INT,  
    OUT p_mensaje VARCHAR  
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
BEGIN  
  
    -- Validación nombre  
  
    IF p_nombre_sucursal IS NULL  
        OR TRIM(p_nombre_sucursal) = '' THEN  
  
        p_codigo := 100;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
        RETURN;  
    END IF;  
  
    -- Validación departamento  
  
    IF NOT EXISTS (
```

```
SELECT 1

FROM departamento

WHERE id_departamento = p_id_departamento

) THEN

    p_codigo := 104;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Insert

INSERT INTO sucursal(

    nombre_sucursal,

    direccion_sucursal,

    id_departamento

)

VALUES(

    TRIM(p_nombre_sucursal),

    TRIM(p_direccion_sucursal),

    p_id_departamento

);

p_codigo := 0;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN unique_violation THEN

        p_codigo := 101;
        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN foreign_key_violation THEN

        p_codigo := 104;
        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN OTHERS THEN

        p_codigo := 500;
        p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_modificar_sucursal(
```

```
IN p_id_sucursal INT,  
IN p_nombre_sucursal VARCHAR,  
IN p_direccion_sucursal VARCHAR,  
IN p_id_departamento INT,  
OUT p_codigo INT,  
OUT p_mensaje VARCHAR  
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
BEGIN  
  
    -- Validación existencia  
  
    IF NOT EXISTS (  
  
        SELECT 1  
  
        FROM sucursal  
  
        WHERE id_sucursal = p_id_sucursal  
  
    ) THEN  
  
        p_codigo := 102;  
  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
        RETURN;  
  
    END IF;
```

```
-- Validación nombre

IF p_nombre_sucursal IS NULL

    OR TRIM(p_nombre_sucursal) = '' THEN

    p_codigo := 100;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación departamento

IF NOT EXISTS (

    SELECT 1

    FROM departamento

    WHERE id_departamento = p_id_departamento

) THEN

    p_codigo := 104;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Update

UPDATE sucursal
```



```
SET nombre_sucursal = TRIM(p_nombre_sucursal),  
    direccion_sucursal = TRIM(p_direccion_sucursal),  
    id_departamento = p_id_departamento  
WHERE id_sucursal = p_id_sucursal;  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION  
  
    WHEN unique_violation THEN  
  
        p_codigo := 101;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
    WHEN foreign_key_violation THEN  
  
        p_codigo := 104;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
    WHEN OTHERS THEN  
  
        p_codigo := 500;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_baja_sucursal(
    IN p_id_sucursal INT,
    OUT p_codigo INT,
    OUT p_mensaje VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN

    -- Validación existencia
    IF NOT EXISTS (
        SELECT 1
        FROM sucursal
        WHERE id_sucursal = p_id_sucursal
    ) THEN

        p_codigo := 102;
        p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
RETURN;  
  
END IF;  
  
-- Delete  
  
DELETE FROM sucursal  
WHERE id_sucursal = p_id_sucursal;  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION  
  
WHEN foreign_key_violation THEN  
  
    p_codigo := 103;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
  
WHEN OTHERS THEN  
  
    p_codigo := 500;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;
```

\$\$;

Cliente

```
CREATE OR REPLACE PROCEDURE pr_login_usuario(  
    IN p_email_ingresado VARCHAR(200),  
    IN p_password_ingresada VARCHAR(200),  
    OUT p_id_usuario INT,  
    OUT p_email_retorno VARCHAR(200),  
    OUT p_id_cliente INT,  
    OUT p_mensaje_resultado VARCHAR(150)  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_password_guardada VARCHAR(200);  
BEGIN  
  
    p_id_usuario      := NULL;  
    p_email_retorno   := NULL;  
    p_id_cliente      := NULL;  
  
    IF TRIM(p_email_ingresado) = '' OR p_email_ingresado IS NULL THEN
```

```
        RAISE EXCEPTION 'El correo electrónico es obligatorio.' USING
ERRCODE = '22000';

    END IF;

    IF TRIM(p_password_ingresada) = '' OR p_password_ingresada IS NULL THEN

        RAISE EXCEPTION 'La contraseña es obligatoria.' USING ERRCODE =
'22000';

    END IF;

    SELECT id_usuario, email, password, id_cliente
    INTO p_id_usuario, p_email_retorno, v_password_guardada, p_id_cliente
    FROM Usuario
    WHERE email = TRIM(p_email_ingresado);

    IF p_id_usuario IS NULL THEN

        RAISE EXCEPTION 'El correo electrónico no se encuentra registrado.'
USING ERRCODE = '02000';

    END IF;

    IF v_password_guardada <> p_password_ingresada THEN

        RAISE EXCEPTION 'Contraseña incorrecta.' USING ERRCODE = '28P01';
```

```
END IF;

p_mensaje_resultado := 'Login exitoso. Bienvenido al sistema.';

COMMIT;

EXCEPTION

    WHEN no_data_found THEN

        ROLLBACK;

        p_id_usuario      := NULL;
        p_email_retorno := NULL;
        p_id_cliente      := NULL;
        p_mensaje_resultado := 'Error: Usuario no encontrado.';

    WHEN invalid_password THEN

        ROLLBACK;

        p_id_usuario      := NULL;
        p_email_retorno := NULL;
        p_id_cliente      := NULL;
```

```
p_mensaje_resultado := 'Error: Contraseña incorrecta.';

WHEN data_exception THEN

    ROLLBACK;

    p_mensaje_resultado := 'Error: Datos de entrada inválidos.';

WHEN OTHERS THEN

    ROLLBACK;

    p_id_usuario      := NULL;

    p_email_retorno   := NULL;

    p_id_cliente      := NULL;

    p_mensaje_resultado := 'Error inesperado: ' || SQLERRM;

END;

$$;

CREATE OR REPLACE PROCEDURE pr_registrar_usuario(

    p_nombre_completo VARCHAR,

    p_dni VARCHAR,

    p_telefono VARCHAR,

    p_email VARCHAR,

    p_password VARCHAR

)
```

```
LANGUAGE plpgsql

AS $$

DECLARE
    v_id_cliente INT;
BEGIN

    INSERT INTO Cliente (nombre_completo, dni, telefono)
    VALUES (p_nombre_completo, p_dni, p_telefono)
    RETURNING id_cliente INTO v_id_cliente;

    INSERT INTO Usuario (email, password, id_cliente)
    VALUES (p_email, p_password, v_id_cliente);

    RAISE NOTICE 'Cliente y Usuario registrados exitosamente.';

EXCEPTION

    WHEN unique_violation THEN

        RAISE EXCEPTION 'Error de duplicidad: El DNI o el Email ya se
encuentran registrados.';
```



```
WHEN OTHERS THEN

    RAISE EXCEPTION 'Error inesperado en la transacción: %', SQLERRM;
END;

CREATE OR REPLACE PROCEDURE actualizar_cliente_y_usuario(
    p_id_cliente INT,
    p_nombre_completo VARCHAR,
    p_dni VARCHAR,
    p_telefono VARCHAR,
    p_email VARCHAR,
    p_nuevo_password VARCHAR DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
BEGIN

    UPDATE Cliente
    SET nombre_completo = p_nombre_completo,
        dni = p_dni,
        telefono = p_telefono
    WHERE id_cliente = p_id_cliente;
```

```
IF NOT FOUND THEN

    RAISE EXCEPTION 'Error: El cliente con ID % no existe.',
p_id_cliente;

END IF;


UPDATE Usuario

SET email = p_email

WHERE id_cliente = p_id_cliente;


IF p_nuevo_password IS NOT NULL AND p_nuevo_password <> '' THEN

    IF length(p_nuevo_password) < 8 THEN

        RAISE EXCEPTION 'Contraseña insegura: Debe tener al menos 8
caracteres.';

    ELSIF p_nuevo_password !~ '[A-Z]' THEN

        RAISE EXCEPTION 'Contraseña insegura: Debe contener al menos una
letra mayúscula.';

    ELSIF p_nuevo_password !~ '[a-z]' THEN

        RAISE EXCEPTION 'Contraseña insegura: Debe contener al menos una
letra minúscula.';

    ELSIF p_nuevo_password !~ '[0-9]' THEN
```

```
RAISE EXCEPTION 'Contraseña insegura: Debe contener al menos un
número.';

END IF;

UPDATE Usuario

SET password = p_nuevo_password

WHERE id_cliente = p_id_cliente;

RAISE NOTICE 'Contraseña actualizada de forma segura.';

END IF;

Commit;

RAISE NOTICE 'Datos del cliente actualizados exitosamente.';

EXCEPTION

    WHEN unique_violation THEN

        RAISE EXCEPTION 'Error de duplicidad: El DNI o Email ya están
registrados por otro usuario.';

    WHEN OTHERS THEN

        RAISE EXCEPTION 'Error al procesar la actualización: %', SQLERRM;

END;

$$;

CREATE OR REPLACE FUNCTION obtener_datos_usuario(p_id_usuario INT)
```

```
RETURNS TABLE (  
    id_usuario INT,  
    email VARCHAR,  
    id_cliente INT,  
    nombre_completo VARCHAR,  
    dni VARCHAR,  
    telefono VARCHAR  
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
BEGIN  
  
    RETURN QUERY  
  
    SELECT  
  
        u.id_usuario,  
        u.email,  
        c.id_cliente,  
        c.nombre_completo,  
        c.dni,  
        c.telefono  
  
    FROM Usuario u  
  
    INNER JOIN Cliente c ON u.id_cliente = c.id_cliente  
  
    WHERE u.id_usuario = p_id_usuario;  
  
END;
```

\$\$;

Tipo Vehículo

```
CREATE OR REPLACE PROCEDURE sp_alta_tipo_vehiculo(  
    IN p_nombre_tipo_vehiculo VARCHAR,  
    OUT p_codigo INTEGER,  
    OUT p_mensaje VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  
    IF p_nombre_tipo_vehiculo IS NULL  
        OR TRIM(p_nombre_tipo_vehiculo) = '' THEN  
  
        p_codigo := 100;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
        RETURN;  
    END IF;  
  
    INSERT INTO tipo_vehiculo(nombre_tipo_vehiculo)  
    VALUES(TRIM(p_nombre_tipo_vehiculo));
```

```
p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN unique_violation THEN

        p_codigo := 101;

        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN OTHERS THEN

        p_codigo := 500;

        p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_modificar_tipo_vehiculo(

    IN p_id_tipo_vehiculo INT,

    IN p_nombre_tipo_vehiculo VARCHAR,

    OUT p_codigo INTEGER,

    OUT p_mensaje VARCHAR

)
```

```
LANGUAGE plpgsql

AS $$

BEGIN

    -- Validación existencia

    IF NOT EXISTS (

        SELECT 1

        FROM tipo_vehiculo

        WHERE id_tipo_vehiculo = p_id_tipo_vehiculo

    ) THEN

        p_codigo := 102;

        p_mensaje := fn_obtener_mensaje(p_codigo);

        RETURN;

    END IF;

    -- Validación nombre

    IF p_nombre_tipo_vehiculo IS NULL

        OR TRIM(p_nombre_tipo_vehiculo) = '' THEN

        p_codigo := 100;

        p_mensaje := fn_obtener_mensaje(p_codigo);

        RETURN;

    END IF;
```

```
-- Update

UPDATE tipo_vehiculo

SET nombre_tipo_vehiculo = TRIM(p_nombre_tipo_vehiculo)

WHERE id_tipo_vehiculo = p_id_tipo_vehiculo;

p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN unique_violation THEN

        p_codigo := 101;

        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN OTHERS THEN

        p_codigo := 500;

        p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_baja_tipo_vehiculo(

    IN p_id_tipo_vehiculo INT,
```



```
OUT p_codigo INTEGER,  
OUT p_mensaje VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  
    -- Validación existencia  
    IF NOT EXISTS (  
        SELECT 1  
        FROM tipo_vehiculo  
        WHERE id_tipo_vehiculo = p_id_tipo_vehiculo  
    ) THEN  
        p_codigo := 102;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
        RETURN;  
    END IF;  
  
    -- Delete  
    DELETE FROM tipo_vehiculo  
    WHERE id_tipo_vehiculo = p_id_tipo_vehiculo;  
  
    p_codigo := 0;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN foreign_key_violation THEN

        p_codigo := 103;

        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN OTHERS THEN

        p_codigo := 500;

        p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;
```

Taller

```
CREATE OR REPLACE PROCEDURE sp_alta_taller(

    IN p_nombre_taller VARCHAR,

    IN p_direccion_taller VARCHAR,

    IN p_telefono_taller VARCHAR,

    IN p_id_departamento INT,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR

)
```

```
LANGUAGE plpgsql

AS $$

BEGIN

    -- Validación nombre

    IF p_nombre_taller IS NULL

        OR TRIM(p_nombre_taller) = '' THEN

        p_codigo := 125;

        p_mensaje := fn_obtener_mensaje(p_codigo);

        RETURN;

    END IF;

    -- Validación departamento

    IF p_id_departamento IS NOT NULL

        AND NOT EXISTS (

            SELECT 1

            FROM departamento

            WHERE id_departamento = p_id_departamento

        ) THEN

        p_codigo := 126;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
RETURN;  
  
END IF;  
  
-- Insert  
  
INSERT INTO taller(  
    nombre_taller,  
    direccion_taller,  
    telefono_taller,  
    id_departamento  
)  
VALUES(  
    TRIM(p_nombre_taller),  
    TRIM(p_direccion_taller),  
    TRIM(p_telefono_taller),  
    p_id_departamento  
);  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION
```

```
WHEN unique_violation THEN

    p_codigo := 101;
    p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN foreign_key_violation THEN

    p_codigo := 104;
    p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN OTHERS THEN

    p_codigo := 500;
    p_mensaje := fn_obtener_mensaje(p_codigo);

END;
$$;

CREATE OR REPLACE PROCEDURE sp_modificar_taller(
    IN p_id_taller INT,
    IN p_nombre_taller VARCHAR,
    IN p_direccion_taller VARCHAR,
```

```
IN p_telefono_taller VARCHAR,  
  
IN p_id_departamento INT,  
  
OUT p_codigo INT,  
  
OUT p_mensaje VARCHAR  
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
BEGIN  
  
    -- Validación existencia  
  
    IF NOT EXISTS (  
  
        SELECT 1  
  
        FROM taller  
  
        WHERE id_taller = p_id_taller  
  
    ) THEN  
  
        p_codigo := 102;  
  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
        RETURN;  
  
    END IF;  
  
    -- Validación nombre
```

```
IF p_nombre_taller IS NULL
    OR TRIM(p_nombre_taller) = '' THEN

    p_codigo := 125;
    p_mensaje := fn_obtener_mensaje(p_codigo);
    RETURN;

END IF;

-- Validación departamento
IF p_id_departamento IS NOT NULL
    AND NOT EXISTS (
        SELECT 1
        FROM departamento
        WHERE id_departamento = p_id_departamento
    ) THEN

    p_codigo := 126;
    p_mensaje := fn_obtener_mensaje(p_codigo);
    RETURN;

END IF;
```

```
-- Update

UPDATE taller

SET nombre_taller = TRIM(p_nombre_taller),
    direccion_taller = TRIM(p_direccion_taller),
    telefono_taller = TRIM(p_telefono_taller),
    id_departamento = p_id_departamento
WHERE id_taller = p_id_taller;

p_codigo := 0;
p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN unique_violation THEN

        p_codigo := 101;
        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN foreign_key_violation THEN

        p_codigo := 104;
        p_mensaje := fn_obtener_mensaje(p_codigo);
```



```
WHEN OTHERS THEN

    p_codigo := 500;

    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_baja_taller(

    IN p_id_taller INT,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR

)

LANGUAGE plpgsql

AS $$

BEGIN

    -- Validación existencia

    IF NOT EXISTS (

        SELECT 1

        FROM taller

        WHERE id_taller = p_id_taller

    ) THEN
```

```
p_codigo := 102;

p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Delete

DELETE FROM taller

WHERE id_taller = p_id_taller;

p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN foreign_key_violation THEN

        p_codigo := 103;

        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN OTHERS THEN
```

```
p_codigo := 500;  
  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;  
$;
```

Vehículo

```
CREATE OR REPLACE PROCEDURE sp_alta_vehiculo(  
    IN p_patente VARCHAR,  
    IN p_marca VARCHAR,  
    IN p_modelo VARCHAR,  
    IN p_detalle_confort VARCHAR,  
    IN p_id_tipo_vehiculo INT,  
    IN p_id_sucursal INT,  
    OUT p_codigo INT,  
    OUT p_mensaje VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_id_vehiculo INT;  
BEGIN  
  
    -- Validación patente
```

```
IF p_patente IS NULL
    OR TRIM(p_patente) = '' THEN

    p_codigo := 105;
    p_mensaje := fn_obtener_mensaje(p_codigo);
    RETURN;
END IF;

-- Validación marca
IF p_marca IS NULL
    OR TRIM(p_marca) = '' THEN

    p_codigo := 106;
    p_mensaje := fn_obtener_mensaje(p_codigo);
    RETURN;
END IF;

-- Validación modelo
IF p_modelo IS NULL
    OR TRIM(p_modelo) = '' THEN

    p_codigo := 107;
    p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
RETURN;

END IF;

-- Validación tipo vehículo

IF NOT EXISTS (

    SELECT 1

    FROM tipo_vehiculo

    WHERE id_tipo_vehiculo = p_id_tipo_vehiculo

) THEN

    p_codigo := 108;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación sucursal

IF NOT EXISTS (

    SELECT 1

    FROM sucursal

    WHERE id_sucursal = p_id_sucursal

) THEN

    p_codigo := 109;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Insert vehículo

INSERT INTO vehiculo(

    patente,

    marca,

    modelo,

    detalle_confort,

    id_tipo_vehiculo,

    id_sucursal

)

VALUES(

    UPPER(TRIM(p_patente)),

    TRIM(p_marca),

    TRIM(p_modelo),

    TRIM(p_detalle_confort),

    p_id_tipo_vehiculo,

    p_id_sucursal

)

RETURNING id_vehiculo

INTO v_id_vehiculo;
```

```
-- Estado inicial del vehículo

INSERT INTO vehiculo_x_estado(

    id_vehiculo,

    id_estado_vehiculo

)

VALUES(

    v_id_vehiculo,

    1

);

-- Operación exitosa

p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN unique_violation THEN

        p_codigo := 101;

        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN foreign_key_violation THEN
```

```
p_codigo := 104;

p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN OTHERS THEN

    p_codigo := 500;

    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_modificar_vehiculo(

    IN p_id_vehiculo INT,

    IN p_patente VARCHAR,

    IN p_marca VARCHAR,

    IN p_modelo VARCHAR,

    IN p_detalle_confort VARCHAR,

    IN p_id_tipo_vehiculo INT,

    IN p_id_sucursal INT,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR

)
```



```
LANGUAGE plpgsql

AS $$

BEGIN

    -- Validación existencia

    IF NOT EXISTS (

        SELECT 1

        FROM vehiculo

        WHERE id_vehiculo = p_id_vehiculo

    ) THEN

        p_codigo := 102;

        p_mensaje := fn_obtener_mensaje(p_codigo);

        RETURN;

    END IF;

    -- Validaciones

    IF p_patente IS NULL

        OR TRIM(p_patente) = '' THEN

        p_codigo := 105;

        p_mensaje := fn_obtener_mensaje(p_codigo);

        RETURN;

    END IF;
```

```
END IF;

IF p_marca IS NULL
    OR TRIM(p_marca) = '' THEN

    p_codigo := 106;
    p_mensaje := fn_obtener_mensaje(p_codigo);
    RETURN;
END IF;

IF p_modelo IS NULL
    OR TRIM(p_modelo) = '' THEN

    p_codigo := 107;
    p_mensaje := fn_obtener_mensaje(p_codigo);
    RETURN;
END IF;

-- Validación tipo vehículo
IF NOT EXISTS (

    SELECT 1

    FROM tipo_vehiculo

    WHERE id_tipo_vehiculo = p_id_tipo_vehiculo
```

```
) THEN

    p_codigo := 108;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación sucursal

IF NOT EXISTS (

    SELECT 1

    FROM sucursal

    WHERE id_sucursal = p_id_sucursal

) THEN

    p_codigo := 109;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Update

UPDATE vehiculo

SET patente = UPPER(TRIM(p_patente)),

    marca = TRIM(p_marca),
```

```
    modelo = TRIM(p_modelo),  
    detalle_confort = TRIM(p_detalle_confort),  
    id_tipo_vehiculo = p_id_tipo_vehiculo,  
    id_sucursal = p_id_sucursal  
WHERE id_vehiculo = p_id_vehiculo;  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION  
  
    WHEN unique_violation THEN  
  
        p_codigo := 101;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
    WHEN foreign_key_violation THEN  
  
        p_codigo := 104;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
    WHEN OTHERS THEN
```

```
p_codigo := 500;

p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_baja_vehiculo(
    IN p_id_vehiculo INT,
    OUT p_codigo INT,
    OUT p_mensaje VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN

    -- Validación existencia
    IF NOT EXISTS (
        SELECT 1
        FROM vehiculo
        WHERE id_vehiculo = p_id_vehiculo
    ) THEN

        p_codigo := 102;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Baja desde el estado

INSERT INTO vehiculo_x_estado(

    id_vehiculo,

    id_estado_vehiculo

)

VALUES(

    p_id_vehiculo,

    5

);

p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN foreign_key_violation THEN

        p_codigo := 103;

        p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
WHEN OTHERS THEN

    p_codigo := 500;

    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;
```

Mantenimiento

```
CREATE OR REPLACE PROCEDURE sp_alta_mantenimiento(

    IN p_fecha_envio DATE,

    IN p_fecha_devolucion DATE,

    IN p_id_vehiculo INT,

    IN p_id_taller INT,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR

)

LANGUAGE plpgsql

AS $$

DECLARE

    v_estado_actual INT;

BEGIN
```

```
-- Validación fecha envío

IF p_fecha_envio IS NULL THEN

    p_codigo := 127;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación fechas

IF p_fecha_devolucion IS NOT NULL

    AND p_fecha_devolucion < p_fecha_envio THEN

    p_codigo := 128;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación vehículo

IF NOT EXISTS (

    SELECT 1
```



```
FROM vehiculo

WHERE id_vehiculo = p_id_vehiculo

) THEN

    p_codigo := 114;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación taller

IF NOT EXISTS (

    SELECT 1

    FROM taller

    WHERE id_taller = p_id_taller

) THEN

    p_codigo := 129;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;
```

```
-- Obtener estado actual vehículo

SELECT id_estado_vehiculo

INTO v_estado_actual

FROM vehiculo_x_estado

WHERE id_vehiculo = p_id_vehiculo

ORDER BY fecha_estado DESC

LIMIT 1;

-- Validar disponible

IF v_estado_actual <> 2 THEN

    p_codigo := 130;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Insert mantenimiento

INSERT INTO mantenimiento(

    fecha_envio,

    fecha_devolucion,

    id_vehiculo,

    id_taller
```



```
)
VALUES(
    p_fecha_envio,
    p_fecha_devolucion,
    p_id_vehiculo,
    p_id_taller
);

-- Cambiar estado vehículo a mantenimiento
-- INSERT INTO vehiculo_x_estado(
--     id_vehiculo,
--     id_estado_vehiculo,
--     fecha_estado
-- )
-- VALUES(
--     p_id_vehiculo,
--     3,
--     CURRENT_TIMESTAMP
-- );

p_codigo := 0;
p_mensaje := fn_obtener_mensaje(p_codigo);
```

EXCEPTION

```
WHEN foreign_key_violation THEN

    p_codigo := 104;

    p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN OTHERS THEN

    p_codigo := 500;

    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_modificar_mantenimiento(
    IN p_id_mantenimiento INT,
    IN p_fecha_envio DATE,
    IN p_fecha_devolucion DATE,
    IN p_id_vehiculo INT,
    IN p_id_taller INT,
    OUT p_codigo INT,
    OUT p_mensaje VARCHAR
```

```
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
BEGIN  
  
    -- Validación existencia  
  
    IF NOT EXISTS (  
  
        SELECT 1  
  
        FROM mantenimiento  
  
        WHERE id_mantenimiento = p_id_mantenimiento  
  
    ) THEN  
  
        p_codigo := 102;  
  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
        RETURN;  
  
    END IF;  
  
    -- Validación fecha envío  
  
    IF p_fecha_envio IS NULL THEN  
  
        p_codigo := 127;  
  
        p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
RETURN;  
  
END IF;  
  
-- Validación fechas  
IF p_fecha_devolucion IS NOT NULL  
    AND p_fecha_devolucion < p_fecha_envio THEN  
  
    p_codigo := 128;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
    RETURN;  
  
END IF;  
  
-- Validación vehículo  
IF NOT EXISTS (  
    SELECT 1  
    FROM vehiculo  
    WHERE id_vehiculo = p_id_vehiculo  
) THEN  
  
    p_codigo := 114;  
    p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
RETURN;

END IF;

-- Validación taller

IF NOT EXISTS (

    SELECT 1

    FROM taller

    WHERE id_taller = p_id_taller

) THEN

    p_codigo := 129;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Update

UPDATE mantenimiento

SET fecha_envio = p_fecha_envio,

    fecha_devolucion = p_fecha_devolucion,

    id_vehiculo = p_id_vehiculo,

    id_taller = p_id_taller
```

```
WHERE id_mantenimiento = p_id_mantenimiento;

-- Si volvió del taller → disponible
IF p_fecha_devolucion IS NOT NULL THEN

    INSERT INTO vehiculo_x_estado(

        id_vehiculo,

        id_estado_vehiculo,

        fecha_estado

    )

    VALUES(

        p_id_vehiculo,

        1,

        CURRENT_TIMESTAMP

    );

END IF;

p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION
```



```
WHEN foreign_key_violation THEN

    p_codigo := 104;

    p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN OTHERS THEN

    p_codigo := 500;

    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_baja_mantenimiento(

    IN p_id_mantenimiento INT,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR

)

LANGUAGE plpgsql

AS $$

BEGIN

    -- Validación existencia
```

```
IF NOT EXISTS (  
    SELECT 1  
    FROM mantenimiento  
    WHERE id_mantenimiento = p_id_mantenimiento  
) THEN  
  
    p_codigo := 102;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
    RETURN;  
  
END IF;  
  
-- Delete  
DELETE FROM mantenimiento  
WHERE id_mantenimiento = p_id_mantenimiento;  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION  
  
    WHEN foreign_key_violation THEN
```

```
p_codigo := 103;

p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN OTHERS THEN

    p_codigo := 500;

    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_finalizar_mantenimiento(

    IN p_id_mantenimiento INT,

    IN p_fecha_devolucion DATE,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR

)

LANGUAGE plpgsql

AS $$

DECLARE

    v_id_vehiculo INT;

    v_fecha_envio DATE;

BEGIN
```

```
-- Validar existencia

IF NOT EXISTS (

    SELECT 1

    FROM mantenimiento

    WHERE id_mantenimiento = p_id_mantenimiento

) THEN

    p_codigo := 102;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validar fecha devolución

IF p_fecha_devolucion IS NULL THEN

    p_codigo := 133;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;
```

```
-- Obtener datos mantenimiento

SELECT

    id_vehiculo,

    fecha_envio

INTO

    v_id_vehiculo,

    v_fecha_envio

FROM mantenimiento

WHERE id_mantenimiento = p_id_mantenimiento;

-- Validar fecha

IF p_fecha_devolucion < v_fecha_envio THEN

    p_codigo := 128;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validar ya finalizado

IF EXISTS (

    SELECT 1

    FROM mantenimiento
```

```
WHERE id_mantenimiento = p_id_mantenimiento

AND fecha_devolucion IS NOT NULL

) THEN

    p_codigo := 134;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Actualizar mantenimiento

UPDATE mantenimiento

SET fecha_devolucion = p_fecha_devolucion

WHERE id_mantenimiento = p_id_mantenimiento;

-- Cambiar estado vehículo a disponible

-- INSERT INTO vehiculo_x_estado(

--     id_vehiculo,

--     id_estado_vehiculo,

--     fecha_estado

-- )

-- VALUES(

--     v_id_vehiculo,
```

```
--      1,  
  
--      CURRENT_TIMESTAMP  
  
-- );  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION  
  
    WHEN OTHERS THEN  
  
        p_codigo := 500;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;  
$;
```

Reserva

```
CREATE OR REPLACE PROCEDURE sp_alta_reserva(  
    IN p_fecha_inicio TIMESTAMP,  
    IN p_fecha_fin TIMESTAMP,  
    IN p_sucursal_retiro INT,  
    IN p_sucursal_devolucion INT,  
    IN p_id_cliente INT,
```

```
IN p_id_vehiculo INT,  
OUT p_codigo INT,  
OUT p_mensaje VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_id_reserva INT;  
BEGIN  
  
    -- Validación fecha inicio  
    IF p_fecha_inicio IS NULL THEN  
  
        p_codigo := 110;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
        RETURN;  
  
    END IF;  
  
    -- Validación fecha fin  
    IF p_fecha_fin IS NULL THEN  
  
        p_codigo := 111;
```



```
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
RETURN;  
  
END IF;  
  
-- Validación rango fechas  
  
IF p_fecha_fin <= p_fecha_inicio THEN  
  
    p_codigo := 112;  
  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
  
    RETURN;  
  
END IF;  
  
-- Validación cliente  
  
IF NOT EXISTS (  
    SELECT 1  
    FROM cliente  
    WHERE id_cliente = p_id_cliente  
) THEN  
  
    p_codigo := 113;  
  
    p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
RETURN;  
  
END IF;  
  
-- Validación vehículo  
IF NOT EXISTS (  
    SELECT 1  
    FROM vehiculo  
    WHERE id_vehiculo = p_id_vehiculo  
) THEN  
  
    p_codigo := 114;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
    RETURN;  
  
END IF;  
  
-- Validación sucursal retiro  
IF NOT EXISTS (  
    SELECT 1  
    FROM sucursal  
    WHERE id_sucursal = p_sucursal_retiro  
) THEN
```

```
p_codigo := 115;

p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Validación sucursal devolución
IF p_sucursal_devolucion IS NOT NULL
    AND NOT EXISTS (
        SELECT 1
        FROM sucursal
        WHERE id_sucursal = p_sucursal_devolucion
    ) THEN

    p_codigo := 116;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación disponibilidad vehículo
IF NOT fn_vehiculo_disponible(
```

```
p_id_vehiculo,  
p_fecha_inicio,  
p_fecha_fin  
)  
THEN  
  
    p_codigo := 117;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
    RETURN;  
  
END IF;  
  
-- Insert  
INSERT INTO reserva(  
    fecha_inicio,  
    fecha_fin,  
    sucursal_retiro,  
    sucursal_devolucion,  
    id_cliente,  
    id_vehiculo  
)  
VALUES(  
    p_fecha_inicio,  
    p_fecha_fin,
```

```
p_sucursal_retiro,  
p_sucursal_devolucion,  
p_id_cliente,  
p_id_vehiculo  
)  
  
RETURNING id_reserva  
  
INTO v_id_reserva;  
  
  
-- Estado inicial de la reserva  
  
INSERT INTO reserva_x_estado(  
    id_reserva,  
    id_estado_reserva  
)  
VALUES(  
    v_id_reserva,  
    1  
)  
);  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION
```

```
WHEN foreign_key_violation THEN

    p_codigo := 104;

    p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN OTHERS THEN

    p_codigo := 500;

    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_modificar_reserva(

    IN p_id_reserva INT,

    IN p_fecha_inicio TIMESTAMP,

    IN p_fecha_fin TIMESTAMP,

    IN p_sucursal_retiro INT,

    IN p_sucursal_devolucion INT,

    IN p_id_cliente INT,

    IN p_id_vehiculo INT,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR
```

```
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
BEGIN  
  
    -- Validación existencia reserva  
  
    IF NOT EXISTS (  
        SELECT 1  
        FROM reserva  
        WHERE id_reserva = p_id_reserva  
    ) THEN  
  
        p_codigo := 102;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
        RETURN;  
  
    END IF;  
  
    -- Validación fecha inicio  
  
    IF p_fecha_inicio IS NULL THEN  
  
        p_codigo := 110;  
        p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
RETURN;  
  
END IF;  
  
-- Validación fecha fin  
IF p_fecha_fin IS NULL THEN  
  
    p_codigo := 111;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
    RETURN;  
  
END IF;  
  
-- Validación rango fechas  
IF p_fecha_fin <= p_fecha_inicio THEN  
  
    p_codigo := 112;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
    RETURN;  
  
END IF;  
  
-- Validación cliente
```



```
IF NOT EXISTS (  
    SELECT 1  
    FROM cliente  
    WHERE id_cliente = p_id_cliente  
) THEN  
  
    p_codigo := 113;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
    RETURN;  
  
END IF;  
  
-- Validación vehículo  
  
IF NOT EXISTS (  
    SELECT 1  
    FROM vehiculo  
    WHERE id_vehiculo = p_id_vehiculo  
) THEN  
  
    p_codigo := 114;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
    RETURN;
```

```
END IF;

-- Validación sucursal retiro

IF NOT EXISTS (

    SELECT 1

    FROM sucursal

    WHERE id_sucursal = p_sucursal_retiro

) THEN

    p_codigo := 115;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación sucursal devolución

IF p_sucursal_devolucion IS NOT NULL

    AND NOT EXISTS (

        SELECT 1

        FROM sucursal

        WHERE id_sucursal = p_sucursal_devolucion

    ) THEN
```

```
p_codigo := 116;

p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Validación disponibilidad vehículo

IF NOT fn_vehiculo_disponible_modificacion(
    p_id_reserva,
    p_id_vehiculo,
    p_fecha_inicio,
    p_fecha_fin
) THEN

    p_codigo := 117;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Update

UPDATE reserva

SET fecha_inicio = p_fecha_inicio,
```

```
        fecha_fin = p_fecha_fin,  
        sucursal_retiro = p_sucursal_retiro,  
        sucursal_devolucion = p_sucursal_devolucion,  
        id_cliente = p_id_cliente,  
        id_vehiculo = p_id_vehiculo  
WHERE id_reserva = p_id_reserva;  
  
        p_codigo := 0;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION  
  
        WHEN foreign_key_violation THEN  
  
                p_codigo := 104;  
                p_mensaje := fn_obtener_mensaje(p_codigo);  
  
        WHEN OTHERS THEN  
  
                p_codigo := 500;  
                p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;
```

```
$$;  
  
CREATE OR REPLACE PROCEDURE sp_baja_reserva(  
    IN p_id_reserva INT,  
    OUT p_codigo INT,  
    OUT p_mensaje VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  
    -- Validación existencia  
    IF NOT EXISTS (  
        SELECT 1  
        FROM reserva  
        WHERE id_reserva = p_id_reserva  
    ) THEN  
  
        p_codigo := 102;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
        RETURN;  
  
    END IF;
```

```
-- Baja de un reserva en el estado

INSERT INTO reserva_x_estado(

    id_reserva,

    id_estado_reserva

)

VALUES(

    p_id_reserva,

    3

);

p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN foreign_key_violation THEN

        p_codigo := 103;

        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN OTHERS THEN
```

```
p_codigo := 500;  
  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;  
$;
```

Alquiler

```
CREATE OR REPLACE PROCEDURE sp_alta_alquiler(  
    IN p_fecha_inicio TIMESTAMP,  
    IN p_fecha_fin_prevista TIMESTAMP,  
    IN p_km_inicio INT,  
    IN p_sucursal_retiro INT,  
    IN p_sucursal_devolucion INT,  
    IN p_id_reserva INT,  
    IN p_id_cliente INT,  
    IN p_id_vehiculo INT,  
    OUT p_codigo INT,  
    OUT p_mensaje VARCHAR  
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
DECLARE  
    v_id_alquiler INT;  
  
BEGIN
```

```
-- Validación fecha inicio

IF p_fecha_inicio IS NULL THEN

    p_codigo := 118;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación fecha fin prevista

IF p_fecha_fin_prevista IS NULL THEN

    p_codigo := 119;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación rango fechas

IF p_fecha_fin_prevista <= p_fecha_inicio THEN

    p_codigo := 120;
```



```
p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Validación kilómetros
IF p_km_inicio IS NULL
  OR p_km_inicio < 0 THEN

  p_codigo := 121;
  p_mensaje := fn_obtener_mensaje(p_codigo);
  RETURN;

END IF;

-- Validación cliente
IF NOT EXISTS (
  SELECT 1
  FROM cliente
  WHERE id_cliente = p_id_cliente
) THEN

  p_codigo := 113;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Validación vehículo
IF NOT EXISTS (
    SELECT 1
    FROM vehiculo
    WHERE id_vehiculo = p_id_vehiculo
) THEN

    p_codigo := 114;
    p_mensaje := fn_obtener_mensaje(p_codigo);
    RETURN;

END IF;

-- Validación sucursal retiro
IF NOT EXISTS (
    SELECT 1
    FROM sucursal
    WHERE id_sucursal = p_sucursal_retiro
```

```
) THEN

    p_codigo := 115;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación sucursal devolución
IF p_sucursal_devolucion IS NOT NULL
    AND NOT EXISTS (
        SELECT 1
        FROM sucursal
        WHERE id_sucursal = p_sucursal_devolucion
    ) THEN

    p_codigo := 116;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación reserva
```

```
IF p_id_reserva IS NOT NULL

    AND NOT EXISTS (

        SELECT 1

        FROM reserva

        WHERE id_reserva = p_id_reserva

    ) THEN

    p_codigo := 122;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación disponibilidad vehículo

IF NOT fn_vehiculo_disponible(

    p_id_vehiculo,

    p_fecha_inicio,

    p_fecha_fin_prevista,

    p_id_reserva

) THEN

    p_codigo := 117;

    p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
RETURN;  
  
END IF;  
  
-- Insert alquiler  
INSERT INTO alquiler(  
    fecha_inicio,  
    fecha_fin_prevista,  
    km_inicio,  
    sucursal_retiro,  
    sucursal_devolucion,  
    id_reserva,  
    id_cliente,  
    id_vehiculo  
)  
VALUES(  
    p_fecha_inicio,  
    p_fecha_fin_prevista,  
    p_km_inicio,  
    p_sucursal_retiro,  
    p_sucursal_devolucion,  
    p_id_reserva,  
    p_id_cliente,
```

```
p_id_vehiculo
)
RETURNING id_alquiler
INTO v_id_alquiler;

--Insert estado
INSERT INTO alquiler_x_estado(
    id_alquiler,
    id_estado_alquiler
)
VALUES(
    v_id_alquiler,
    1
);

-- Cambiar estado vehículo
-- INSERT INTO vehiculo_x_estado(
--     id_vehiculo,
--     id_estado_vehiculo,
--     fecha_estado
-- )
-- VALUES(
--     p_id_vehiculo,
```

```
--      3,  
  
--      CURRENT_TIMESTAMP  
  
-- );  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION  
  
    WHEN foreign_key_violation THEN  
  
        p_codigo := 104;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
    WHEN OTHERS THEN  
  
        p_codigo := 500;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;  
$$;  
  
CREATE OR REPLACE PROCEDURE sp_modificar_alquiler(
```

```
IN p_id_alquiler INT,  
IN p_fecha_inicio TIMESTAMP,  
IN p_fecha_fin_prevista TIMESTAMP,  
IN p_fecha_fin_real TIMESTAMP,  
IN p_km_inicio INT,  
IN p_km_fin INT,  
IN p_sucursal_retiro INT,  
IN p_sucursal_devolucion INT,  
IN p_id_cliente INT,  
IN p_id_vehiculo INT,  
OUT p_codigo INT,  
OUT p_mensaje VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  
    -- Validación existencia  
    IF NOT EXISTS (  
        SELECT 1  
        FROM alquiler  
        WHERE id_alquiler = p_id_alquiler  
    ) THEN
```



```
p_codigo := 102;

p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Validación rango fechas

IF p_fecha_fin_prevista <= p_fecha_inicio THEN

    p_codigo := 120;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación km final

IF p_km_fin IS NOT NULL

    AND p_km_fin < p_km_inicio THEN

    p_codigo := 123;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;
```

```
END IF;

-- Update
UPDATE alquiler
SET fecha_inicio = p_fecha_inicio,
    fecha_fin_prevista = p_fecha_fin_prevista,
    fecha_fin_real = p_fecha_fin_real,
    km_inicio = p_km_inicio,
    km_fin = p_km_fin,
    sucursal_retiro = p_sucursal_retiro,
    sucursal_devolucion = p_sucursal_devolucion,
    id_cliente = p_id_cliente,
    id_vehiculo = p_id_vehiculo
WHERE id_alquiler = p_id_alquiler;

-- Si finalizó → vehículo disponible
IF p_fecha_fin_real IS NOT NULL THEN

    INSERT INTO vehiculo_x_estado(
        id_vehiculo,
        id_estado_vehiculo,
        fecha_estado
```

```
)  
  
VALUES(  
  
    p_id_vehiculo,  
  
    1,  
  
    CURRENT_TIMESTAMP  
  
);  
  
  
END IF;  
  
  
p_codigo := 0;  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
EXCEPTION  
  
  
WHEN OTHERS THEN  
  
  
    p_codigo := 500;  
    p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;  
$$;  
  
CREATE OR REPLACE PROCEDURE sp_baja_alquiler(  

```

```
IN p_id_alquiler INT,  
OUT p_codigo INT,  
OUT p_mensaje VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_id_vehiculo INT;  
BEGIN  
  
    -- Validación existencia  
    IF NOT EXISTS (  
        SELECT 1  
        FROM alquiler  
        WHERE id_alquiler = p_id_alquiler  
    ) THEN  
  
        p_codigo := 102;  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
        RETURN;  
  
    END IF;
```

```
-- Obtener vehículo

SELECT id_vehiculo
INTO v_id_vehiculo
FROM alquiler
WHERE id_alquiler = p_id_alquiler;

-- Delete

DELETE FROM alquiler
WHERE id_alquiler = p_id_alquiler;

-- Vehículo disponible

INSERT INTO vehiculo_x_estado(
    id_vehiculo,
    id_estado_vehiculo,
    fecha_estado
)
VALUES(
    v_id_vehiculo,
    1,
    CURRENT_TIMESTAMP
);

p_codigo := 0;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN foreign_key_violation THEN

        p_codigo := 103;
        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN OTHERS THEN

        p_codigo := 500;
        p_mensaje := fn_obtener_mensaje(p_codigo);

END;
$;
```

Tarifa

```
CREATE OR REPLACE PROCEDURE sp_alta_tarifa(
    IN p_precio_dia NUMERIC,
    IN p_porcentaje_recargo_hora NUMERIC,
    IN p_id_sucursal INT,
    IN p_id_tipo_vehiculo INT,
    OUT p_codigo INT,
```

```
OUT p_mensaje VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN

    -- Validación precio día
    IF p_precio_dia IS NULL
        OR p_precio_dia <= 0 THEN

        p_codigo := 131;
        p_mensaje := fn_obtener_mensaje(p_codigo);
        RETURN;

    END IF;

    -- Validación recargo
    IF p_porcentaje_recargo_hora IS NULL
        OR p_porcentaje_recargo_hora < 0 THEN

        p_codigo := 132;
        p_mensaje := fn_obtener_mensaje(p_codigo);
        RETURN;
```

```
END IF;

-- Validación sucursal

IF NOT EXISTS (

    SELECT 1

    FROM sucursal

    WHERE id_sucursal = p_id_sucursal

) THEN

    p_codigo := 109;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación tipo vehículo

IF NOT EXISTS (

    SELECT 1

    FROM tipo_vehiculo

    WHERE id_tipo_vehiculo = p_id_tipo_vehiculo

) THEN
```



```
p_codigo := 108;

p_mensaje := fn_obtener_mensaje(p_codigo);

RETURN;

END IF;

-- Insert

INSERT INTO tarifa(

    precio_dia,

    porcentaje_recargo_hora,

    id_sucursal,

    id_tipo_vehiculo

)

VALUES(

    p_precio_dia,

    p_porcentaje_recargo_hora,

    p_id_sucursal,

    p_id_tipo_vehiculo

);

p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);
```

EXCEPTION

```
WHEN unique_violation THEN

    p_codigo := 101;
    p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN foreign_key_violation THEN

    p_codigo := 104;
    p_mensaje := fn_obtener_mensaje(p_codigo);

WHEN OTHERS THEN

    p_codigo := 500;
    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_modificar_tarifa(
    IN p_id_tarifa INT,
    IN p_precio_dia NUMERIC,
```

```
IN p_porcentaje_recargo_hora NUMERIC,  
  
IN p_id_sucursal INT,  
  
IN p_id_tipo_vehiculo INT,  
  
OUT p_codigo INT,  
  
OUT p_mensaje VARCHAR  
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
BEGIN  
  
    -- Validación existencia  
  
    IF NOT EXISTS (  
  
        SELECT 1  
  
        FROM tarifa  
  
        WHERE id_tarifa = p_id_tarifa  
  
    ) THEN  
  
        p_codigo := 102;  
  
        p_mensaje := fn_obtener_mensaje(p_codigo);  
  
        RETURN;  
  
    END IF;
```

```
-- Validación precio día

IF p_precio_dia IS NULL

    OR p_precio_dia <= 0 THEN

    p_codigo := 131;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación recargo

IF p_porcentaje_recargo_hora IS NULL

    OR p_porcentaje_recargo_hora < 0 THEN

    p_codigo := 132;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación sucursal

IF NOT EXISTS (

    SELECT 1
```

```
FROM sucursal

WHERE id_sucursal = p_id_sucursal

) THEN

    p_codigo := 109;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validación tipo vehículo

IF NOT EXISTS (

    SELECT 1

    FROM tipo_vehiculo

    WHERE id_tipo_vehiculo = p_id_tipo_vehiculo

) THEN

    p_codigo := 108;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;
```

```
-- Update

UPDATE tarifa

SET precio_dia = p_precio_dia,

    porcentaje_recargo_hora = p_porcentaje_recargo_hora,

    id_sucursal = p_id_sucursal,

    id_tipo_vehiculo = p_id_tipo_vehiculo

WHERE id_tarifa = p_id_tarifa;

p_codigo := 0;

p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN unique_violation THEN

        p_codigo := 101;

        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN foreign_key_violation THEN

        p_codigo := 104;

        p_mensaje := fn_obtener_mensaje(p_codigo);
```

```
WHEN OTHERS THEN

    p_codigo := 500;

    p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

CREATE OR REPLACE PROCEDURE sp_baja_tarifa(

    IN p_id_tarifa INT,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR

)

LANGUAGE plpgsql

AS $$

BEGIN

    -- Validación existencia

    IF NOT EXISTS (

        SELECT 1

        FROM tarifa

        WHERE id_tarifa = p_id_tarifa

    ) THEN
```

```
        p_codigo := 102;

        p_mensaje := fn_obtener_mensaje(p_codigo);

        RETURN;

    END IF;

    -- Delete

    DELETE FROM tarifa

    WHERE id_tarifa = p_id_tarifa;

    p_codigo := 0;

    p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN foreign_key_violation THEN

        p_codigo := 103;

        p_mensaje := fn_obtener_mensaje(p_codigo);

    WHEN OTHERS THEN
```



```
p_codigo := 500;  
  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;  
$$;
```

Finalizar Alquiler y Migrar desde reserva

```
CREATE OR REPLACE PROCEDURE sp_finalizar_alquiler(  
    IN p_id_alquiler INT,  
    IN p_fecha_fin_real TIMESTAMP,  
    IN p_km_fin INT,  
    IN p_sucursal_devolucion INT,  
    OUT p_codigo INT,  
    OUT p_mensaje VARCHAR  
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
DECLARE  
  
    v_fecha_inicio TIMESTAMP;  
    v_fecha_fin_prevista TIMESTAMP;  
    v_km_inicio INT;
```

```
v_id_vehiculo INT;

v_id_factura INT;


v_id_tipo_vehiculo INT;


v_precio_dia NUMERIC(10,2);
v_recargo_hora NUMERIC(10,2);


v_dias_alquiler INT;
v_horas_recargo INT;


v_monto_base NUMERIC(10,2);
v_monto_recargo NUMERIC(10,2);
v_total NUMERIC(10,2);


v_estado_actual INT;
BEGIN

-- Validar existencia alquiler
IF NOT EXISTS (

    SELECT 1

    FROM alquiler

    WHERE id_alquiler = p_id_alquiler
```

```
) THEN

    p_codigo := 102;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Validar fecha fin real
IF p_fecha_fin_real IS NULL THEN

    p_codigo := 135;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Obtener alquiler

SELECT

    fecha_inicio,

    fecha_fin_prevista,

    km_inicio,

    id_vehiculo
```

```
INTO

    v_fecha_inicio,

    v_fecha_fin_prevista,

    v_km_inicio,

    v_id_vehiculo

FROM alquiler

WHERE id_alquiler = p_id_alquiler;

-- Validar km

IF p_km_fin < v_km_inicio THEN

    p_codigo := 136;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Estado actual alquiler

SELECT id_estado_alquiler

INTO v_estado_actual

FROM alquiler_x_estado

WHERE id_alquiler = p_id_alquiler

ORDER BY fecha_estado DESC
```

```
LIMIT 1;

-- Validar activo

IF v_estado_actual <> 1 THEN

    p_codigo := 137;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Obtener tipo vehículo

SELECT id_tipo_vehiculo

INTO v_id_tipo_vehiculo

FROM vehiculo

WHERE id_vehiculo = v_id_vehiculo;

-- Obtener tarifa

SELECT

    precio_dia,

    porcentaje_recargo_hora

INTO

    v_precio_dia,
```

```
v_recargo_hora

FROM tarifa

WHERE id_tipo_vehiculo = v_id_tipo_vehiculo

AND id_sucursal = p_sucursal_devolucion

LIMIT 1;

-- Validar tarifa

IF v_precio_dia IS NULL THEN

    p_codigo := 138;

    p_mensaje := fn_obtener_mensaje(p_codigo);

    RETURN;

END IF;

-- Calcular días

v_dias_alquiler := CEIL(

    EXTRACT(EPOCH FROM (

        p_fecha_fin_real - v_fecha_inicio

    )) / 86400

);

IF v_dias_alquiler <= 0 THEN
```

```
v_dias_alquiler := 1;

END IF;

-- Monto base
v_monto_base := v_dias_alquiler * v_precio_dia;

-- Horas atraso
IF p_fecha_fin_real > v_fecha_fin_prevista THEN

    v_horas_recargo := CEIL(
        EXTRACT(EPOCH FROM (
            p_fecha_fin_real - v_fecha_fin_prevista
        )) / 3600
    );

ELSE

    v_horas_recargo := 0;

END IF;

-- Recargo
v_monto_recargo :=
```

```
(v_precio_dia * (v_recargo_hora / 100))

* v_horas_recargo;

-- Total

v_total := v_monto_base + v_monto_recargo;

-- Actualizar alquiler

UPDATE alquiler

SET fecha_fin_real = p_fecha_fin_real,

    km_fin = p_km_fin,

    sucursal_devolucion = p_sucursal_devolucion

WHERE id_alquiler = p_id_alquiler;

-- Estado finalizado

INSERT INTO alquiler_x_estado(

    id_alquiler,

    id_estado_alquiler,

    fecha_estado

)

VALUES(

    p_id_alquiler,

    2,

    CURRENT_TIMESTAMP
```



```
);  
  
-- Vehículo disponible  
-- INSERT INTO vehiculo_x_estado(  
--     id_vehiculo,  
--     id_estado_vehiculo,  
--     fecha_estado  
-- )  
-- VALUES(  
--     v_id_vehiculo,  
--     1,  
--     CURRENT_TIMESTAMP  
-- );
```

```
-- Crear factura  
INSERT INTO factura(  
    fecha_emision,  
    monto_base,  
    monto_recargo,  
    total,  
    id_alquiler  
)  
VALUES(
```



UNIVERSIDAD
GASTÓN DACHARY

GASTÓN DACHARY

```

        CURRENT_TIMESTAMP,
        v_monto_base,
        v_monto_recargo,
        v_total,
        p_id_alquiler
    )
RETURNING id_factura
INTO v_id_factura;

-- Detalle base
INSERT INTO detalle_factura(
    codigo,
    descripcion,
    precio_unitario,
    subtotal,
    id_factura
)
VALUES(
    1,
    'Alquiler de vehículo',
    v_precio_dia,
    v_monto_base,
    v_id_factura
);

```

```
);

-- Detalle recargo

IF v_monto_recargo > 0 THEN

    INSERT INTO detalle_factura(

        codigo,

        descripcion,

        precio_unitario,

        subtotal,

        id_factura

    )

    VALUES(

        2,

        'Recargo por atraso',

        v_monto_recargo,

        v_monto_recargo,

        v_id_factura

    );

END IF;

p_codigo := 0;
```

```
p_mensaje := fn_obtener_mensaje(p_codigo);

EXCEPTION

    WHEN OTHERS THEN

        p_codigo := 500;

        p_mensaje := fn_obtener_mensaje(p_codigo);

END;

$$;

-- Reserva a Alquiler

CREATE OR REPLACE PROCEDURE sp_generar_alquiler_desde_reserva(

    IN p_id_reserva INT,

    IN p_km_inicio INT,

    OUT p_codigo INT,

    OUT p_mensaje VARCHAR

)

LANGUAGE plpgsql

AS $$

DECLARE

    v_fecha_inicio TIMESTAMP;
```

```
v_fecha_fin TIMESTAMP;

v_sucursal_retiro INT;

v_sucursal_devolucion INT;

v_id_cliente INT;

v_id_vehiculo INT;


v_estado_reserva INT;

BEGIN

    -- Validar existencia reserva

    IF NOT EXISTS (

        SELECT 1

        FROM reserva

        WHERE id_reserva = p_id_reserva

    ) THEN

        p_codigo := 102;

        p_mensaje := fn_obtener_mensaje(p_codigo);

        RETURN;

    END IF;

    -- Estado actual reserva
```

```
SELECT id_estado_reserva
INTO v_estado_reserva
FROM reserva_x_estado
WHERE id_reserva = p_id_reserva
ORDER BY fecha_estado DESC
LIMIT 1;

-- Validar pendiente/confirmada
IF v_estado_reserva <> 1 THEN

    p_codigo := 139;
    p_mensaje := fn_obtener_mensaje(p_codigo);
    RETURN;

END IF;

-- Obtener datos reserva
SELECT
    fecha_inicio,
    fecha_fin,
    sucursal_retiro,
    sucursal_devolucion,
    id_cliente,
```

```
id_vehiculo

INTO

    v_fecha_inicio,
    v_fecha_fin,
    v_sucursal_retiro,
    v_sucursal_devolucion,
    v_id_cliente,
    v_id_vehiculo

FROM reserva

WHERE id_reserva = p_id_reserva;

-- Reutilizar alta alquiler

CALL sp_alta_alquiler(

    v_fecha_inicio,
    v_fecha_fin,
    p_km_inicio,
    v_sucursal_retiro,
    v_sucursal_devolucion,
    p_id_reserva,
    v_id_cliente,
    v_id_vehiculo,
    p_codigo,
    p_mensaje
```

```
);

-- Si falló
IF p_codigo <> 0 THEN
    RETURN;
END IF;

-- Cambiar estado reserva → finalizada/usada
INSERT INTO reserva_x_estado(
    id_reserva,
    id_estado_reserva,
    fecha_estado
)
VALUES(
    p_id_reserva,
    2,
    CURRENT_TIMESTAMP
);

EXCEPTION

    WHEN OTHERS THEN
```



```
p_codigo := 500;  
  
p_mensaje := fn_obtener_mensaje(p_codigo);  
  
END;  
$;
```

Secuencias

```
CREATE SEQUENCE seq_log_sucursal  
START 1  
INCREMENT 1;  
  
CREATE SEQUENCE seq_log_tipo_vehiculo  
START 1  
INCREMENT 1;  
  
CREATE SEQUENCE seq_log_vehiculo  
START 1  
INCREMENT 1;  
  
CREATE SEQUENCE seq_log_reserva  
START 1  
INCREMENT 1;
```

```
CREATE SEQUENCE seq_log_alquiler
START 1
INCREMENT 1;

CREATE SEQUENCE seq_log_alquiler_x_estado
START 1
INCREMENT 1;

CREATE SEQUENCE seq_log_reserva_x_estado
START 1
INCREMENT 1;

CREATE SEQUENCE seq_log_vehiculo_x_estado
START 1
INCREMENT 1;

CREATE SEQUENCE seq_log_detalle_factura
START 1
INCREMENT 1;

CREATE SEQUENCE seq_log_factura
START 1
INCREMENT 1;
```

```
CREATE SEQUENCE seq_log_taller  
  
START 1  
  
INCREMENT 1;  
  
CREATE SEQUENCE seq_log_mantenimiento  
  
START 1  
  
INCREMENT 1;  
  
CREATE SEQUENCE seq_log_tarifa  
  
START 1  
  
INCREMENT 1;
```

Triggers

```
CREATE OR REPLACE FUNCTION fn_log_sucursal()  
  
RETURNS TRIGGER  
  
LANGUAGE plpgsql  
  
AS $$  
  
DECLARE  
  
    v_usuario INT;  
  
    v_id_log INT;  
  
    v_usuario_db VARCHAR(50);  
  
BEGIN
```

```
-- Usuario PostgreSQL

v_usuario_db := current_user;

-- Usuario enviado desde Laravel

v_usuario := current_setting('app.usuario', true)::INT;

-- INSERT

IF TG_OP = 'INSERT' THEN

    v_id_log := nextval('seq_log_sucursal');

    INSERT INTO log_sucursal (

        id_log,

        id_sucursal,

        nombre_sucursal,

        direccion_sucursal,

        id_departamento,

        mov,

        usuario,

        usuario_db

    )

    VALUES (
```



**UNIVERSIDAD
GASTÓN DACHARY**

```

        v_id_log,

        NEW.id_sucursal,

        NEW.nombre_sucursal,

        NEW.direccion_sucursal,

        NEW.id_departamento,

        'I',

        v_usuario,

        v_usuario_db

    );

RETURN NEW;

-- UPDATE

ELSIF TG_OP = 'UPDATE' THEN

    -- Antes del update

    v_id_log := nextval('seq_log_sucursal');

    INSERT INTO log_sucursal (

        id_log,

        id_sucursal,

        nombre_sucursal,

        direccion_sucursal,

```



```
id_departamento,
mov,
usuario,
usuario_db
)
VALUES (
v_id_log,
OLD.id_sucursal,
OLD.nombre_sucursal,
OLD.direccion_sucursal,
OLD.id_departamento,
'AU',
v_usuario,
v_usuario_db
);

-- Después del update
v_id_log := nextval('seq_log_sucursal');

INSERT INTO log_sucursal (
id_log,
id_sucursal,
nombre_sucursal,
```

mov,

usuario,

```
usuario_db
```

)

VALUES (

v_id_log,

```
OLD.id_sucursal,
```

OLD.nombre_sucursal,

OLD.direccion_sucursal,

OLD.id_departamento,

'AU',

v_usuario,

```
v_usuario_db
```

);

-- Después del update

```
v_id_log := nextval('seq_log_sucursal');
```

```
INSERT INTO log_sucursal (
```

```
id_log,
```

```
id_sucursal,
```

```
nombre_sucursal,
```

```
        direccion_sucursal,  
        id_departamento,  
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES (  
    v_id_log,  
    NEW.id_sucursal,  
    NEW.nombre_sucursal,  
    NEW.direccion_sucursal,  
    NEW.id_departamento,  
    'DU',  
    v_usuario,  
    v_usuario_db  
);  
  
RETURN NEW;  
  
-- DELETE  
ELSIF TG_OP = 'DELETE' THEN  
  
    v_id_log := nextval('seq_log_sucursal');
```

```
INSERT INTO log_sucursal (  
    id_log,  
    id_sucursal,  
    nombre_sucursal,  
    direccion_sucursal,  
    id_departamento,  
    mov,  
    usuario,  
    usuario_db  
)  
VALUES (  
    v_id_log,  
    OLD.id_sucursal,  
    OLD.nombre_sucursal,  
    OLD.direccion_sucursal,  
    OLD.id_departamento,  
    'D',  
    v_usuario,  
    usuario_db  
);  
  
RETURN OLD;
```



```
END IF;

RETURN NULL;

END;

$$;

CREATE TRIGGER tr_log_sucursal
AFTER INSERT OR UPDATE OR DELETE
ON sucursal
FOR EACH ROW
EXECUTE FUNCTION fn_log_sucursal();
-----
-- Tipo Vehiculo
CREATE OR REPLACE FUNCTION fn_log_tipo_vehiculo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_usuario INT;
    v_id_log INT;
    v_usuario_db VARCHAR(50);
BEGIN
```

```
-- Usuario PostgreSQL

v_usuario_db := current_user;

-- Obtener usuario desde Laravel

v_usuario := current_setting('app.usuario', true)::INT;

-- Obtener ID del Log

v_id_log := nextval('seq_log_tipo_vehiculo');

-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_tipo_vehiculo (

        id_log,

        id_tipo_vehiculo,

        nombre_tipo_vehiculo,

        mov,

        usuario,

        usuario_db

    )

    VALUES (

        v_id_log,
```



```
VALUES (  
    v_id_log,  
    OLD.id_tipo_vehiculo,  
    OLD.nombre_tipo_vehiculo,  
    'AU',  
    v_usuario,  
    v_usuario_db  
);  
  
-- Después del update  
v_id_log := nextval('seq_log_tipo_vehiculo');  
  
INSERT INTO log_tipo_vehiculo (  
    id_log,  
    id_tipo_vehiculo,  
    nombre_tipo_vehiculo,  
    mov,  
    usuario,  
    usuario_db  
)  
VALUES (  
    v_id_log,  
    NEW.id_tipo_vehiculo,
```

```
NEW.nombre_tipo_vehiculo,  
  
    'DU',  
  
    v_usuario,  
  
    v_usuario_db  
  
);  
  
RETURN NEW;  
  
  
-- DELETE  
  
ELSIF TG_OP = 'DELETE' THEN  
  
    INSERT INTO log_tipo_vehiculo (  
  
        id_log,  
  
        id_tipo_vehiculo,  
  
        nombre_tipo_vehiculo,  
  
        mov,  
  
        usuario,  
  
        usuario_db  
  
    )  
  
    VALUES (  
  
        v_id_log,  
  
        OLD.id_tipo_vehiculo,  
  
        OLD.nombre_tipo_vehiculo,
```

```
'D',  
  
v_usuario,  
  
v_usuario_db  
  
);  
  
RETURN OLD;  
  
END IF;  
  
RETURN NULL;  
END;  
$$;  
  
CREATE TRIGGER tr_log_tipo_vehiculo  
AFTER INSERT OR UPDATE OR DELETE  
ON tipo_vehiculo  
FOR EACH ROW  
EXECUTE FUNCTION fn_log_tipo_vehiculo();  
  
-----  
  
CREATE OR REPLACE FUNCTION fn_log_vehiculo()  
RETURNS TRIGGER  
LANGUAGE plpgsql
```

```
AS $$  
  
DECLARE  
  
    v_usuario INT;  
  
    v_id_log INT;  
  
    v_usuario_db VARCHAR(50);  
  
BEGIN  
  
    -- Usuario PostgreSQL  
  
    v_usuario_db := current_user;  
  
    -- Usuario enviado desde Laravel  
  
    v_usuario := current_setting('app.usuario', true)::INT;  
  
    -- INSERT  
  
    IF TG_OP = 'INSERT' THEN  
  
        v_id_log := nextval('seq_log_vehiculo');  
  
        INSERT INTO log_vehiculo (  
            id_log,  
            id_vehiculo,  
            patente,  
            marca,
```

```
        modelo,  
        detalle_confort,  
        id_tipo_vehiculo,  
        id_sucursal,  
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES (  
    v_id_log,  
    NEW.id_vehiculo,  
    NEW.patente,  
    NEW.marca,  
    NEW.modelo,  
    NEW.detalle_confort,  
    NEW.id_tipo_vehiculo,  
    NEW.id_sucursal,  
    'I',  
    v_usuario,  
    v_usuario_db  
);  
  
RETURN NEW;
```



```
-- UPDATE

ELSIF TG_OP = 'UPDATE' THEN

    -- Antes del update

    v_id_log := nextval('seq_log_vehiculo');

    INSERT INTO log_vehiculo (

        id_log,

        id_vehiculo,

        patente,

        marca,

        modelo,

        detalle_confort,

        id_tipo_vehiculo,

        id_sucursal,

        mov,

        usuario,

        usuario_db

    )

    VALUES (

        v_id_log,

        OLD.id_vehiculo,
```

```
        OLD.patente,  
        OLD.marca,  
        OLD.modelo,  
        OLD.detalle_comfort,  
        OLD.id_tipo_vehiculo,  
        OLD.id_sucursal,  
        'AU',  
        v_usuario,  
        v_usuario_db  
    );  
  
    -- Después del update  
    v_id_log := nextval('seq_log_vehiculo');  
  
    INSERT INTO log_vehiculo (  
        id_log,  
        id_vehiculo,  
        patente,  
        marca,  
        modelo,  
        detalle_comfort,  
        id_tipo_vehiculo,  
        id_sucursal,
```

```
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES (  
    v_id_log,  
    NEW.id_vehiculo,  
    NEW.patente,  
    NEW.marca,  
    NEW.modelo,  
    NEW.detalle_confort,  
    NEW.id_tipo_vehiculo,  
    NEW.id_sucursal,  
    'DU',  
    v_usuario,  
    v_usuario_db  
);  
  
RETURN NEW;  
  
-- DELETE  
ELSIF TG_OP = 'DELETE' THEN
```

```
v_id_log := nextval('seq_log_vehiculo');
```

```
INSERT INTO log_vehiculo (
```

```
    id_log,
```

```
    id_vehiculo,
```

```
    patente,
```

```
    marca,
```

```
    modelo,
```

```
    detalle_confort,
```

```
    id_tipo_vehiculo,
```

```
    id_sucursal,
```

```
    mov,
```

```
    usuario,
```

```
    usuario_db
```

```
)
```

```
VALUES (
```

```
    v_id_log,
```

```
    OLD.id_vehiculo,
```

```
    OLD.patente,
```

```
    OLD.marca,
```

```
    OLD.modelo,
```

```
    OLD.detalle_confort,
```

```
    OLD.id_tipo_vehiculo,
```

```
        OLD.id_sucursal,  
        'D',  
        v_usuario,  
        v_usuario_db  
    );  
  
    RETURN OLD;  
  
END IF;  
  
RETURN NULL;  
END;  
$$;  
  
CREATE TRIGGER tr_log_vehiculo  
AFTER INSERT OR UPDATE OR DELETE  
ON vehiculo  
FOR EACH ROW  
EXECUTE FUNCTION fn_log_vehiculo();  
  
-- Reserva  
CREATE OR REPLACE FUNCTION fn_log_reserva()  
RETURNS TRIGGER
```

```
LANGUAGE plpgsql

AS $$

DECLARE

    v_usuario INT;

    v_id_log INT;

    v_usuario_db VARCHAR(50);

BEGIN

    -- Usuario PostgreSQL

    v_usuario_db := current_user;

    -- Usuario desde Laravel

    v_usuario := current_setting('app.usuario', true)::INT;

    -- ID Log

    v_id_log := nextval('seq_log_reserva');

    -- INSERT

    IF TG_OP = 'INSERT' THEN

        INSERT INTO log_reserva (

            id_log,

            id_reserva,
```

```
        fecha_inicio,  
        fecha_fin,  
        sucursal_retiro,  
        sucursal_devolucion,  
        id_cliente,  
        id_vehiculo,  
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES (  
    v_id_log,  
    NEW.id_reserva,  
    NEW.fecha_inicio,  
    NEW.fecha_fin,  
    NEW.sucursal_retiro,  
    NEW.sucursal_devolucion,  
    NEW.id_cliente,  
    NEW.id_vehiculo,  
    'I',  
    v_usuario,  
    v_usuario_db  
);
```

```
RETURN NEW;

-- UPDATE

ELSIF TG_OP = 'UPDATE' THEN

    -- Antes Update

    INSERT INTO log_reserva (

        id_log,

        id_reserva,

        fecha_inicio,

        fecha_fin,

        sucursal_retiro,

        sucursal_devolucion,

        id_cliente,

        id_vehiculo,

        mov,

        usuario,

        usuario_db

    )

    VALUES (

        nextval('seq_log_reserva'),

        OLD.id_reserva,
```



```
OLD.fecha_inicio,  
OLD.fecha_fin,  
OLD.sucursal_retiro,  
OLD.sucursal_devolucion,  
OLD.id_cliente,  
OLD.id_vehiculo,  
  
    'AU',  
  
    v_usuario,  
  
    v_usuario_db  
  
);
```

-- Después Update

```
INSERT INTO log_reserva (  
    id_log,  
    id_reserva,  
    fecha_inicio,  
    fecha_fin,  
    sucursal_retiro,  
    sucursal_devolucion,  
    id_cliente,  
    id_vehiculo,  
    mov,  
    usuario,
```

```
        usuario_db
    )
VALUES (
    nextval('seq_log_reserva'),
    NEW.id_reserva,
    NEW.fecha_inicio,
    NEW.fecha_fin,
    NEW.sucursal_retiro,
    NEW.sucursal_devolucion,
    NEW.id_cliente,
    NEW.id_vehiculo,
    'DU',
    v_usuario,
    v_usuario_db
);

RETURN NEW;

-- DELETE
ELSIF TG_OP = 'DELETE' THEN

    INSERT INTO log_reserva (
        id_log,
```

```
id_reserva,  
fecha_inicio,  
fecha_fin,  
sucursal_retiro,  
sucursal_devolucion,  
id_cliente,  
id_vehiculo,  
mov,  
usuario,  
usuario_db  
)  
VALUES (  
    v_id_log,  
    OLD.id_reserva,  
    OLD.fecha_inicio,  
    OLD.fecha_fin,  
    OLD.sucursal_retiro,  
    OLD.sucursal_devolucion,  
    OLD.id_cliente,  
    OLD.id_vehiculo,  
    'D',  
    v_usuario,  
    v_usuario_db
```

```
);

RETURN OLD;

END IF;

RETURN NULL;

END;
$$;

CREATE TRIGGER tr_log_reserva
AFTER INSERT OR UPDATE OR DELETE
ON reserva
FOR EACH ROW
EXECUTE FUNCTION fn_log_reserva();

-- Alquiler
-- Función trigger

CREATE OR REPLACE FUNCTION fn_log_alquiler()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
```

```
v_usuario_app INT;  
  
v_usuario_db VARCHAR(50);  
  
v_id_log INT;  
  
BEGIN  
  
    -- Usuario PostgreSQL  
  
    v_usuario_db := current_user;  
  
    -- Usuario Laravel  
  
BEGIN  
  
    v_usuario_app := current_setting('app.usuario', true)::INT;  
  
EXCEPTION  
  
    WHEN OTHERS THEN  
  
        v_usuario_app := NULL;  
  
END;  
  
    -- ID Log  
  
    v_id_log := nextval('seq_log_alquiler');  
  
    -- INSERT  
  
    IF TG_OP = 'INSERT' THEN  
  
        INSERT INTO log_alquiler(  

```

```
id_log,  
id_alquiler,  
fecha_inicio,  
fecha_fin_prevista,  
fecha_fin_real,  
km_inicio,  
km_fin,  
sucursal_retiro,  
sucursal_devolucion,  
id_reserva,  
id_cliente,  
id_vehiculo,  
mov,  
usuario,  
usuario_db  
)  
VALUES(  
v_id_log,  
NEW.id_alquiler,  
NEW.fecha_inicio,  
NEW.fecha_fin_prevista,  
NEW.fecha_fin_real,  
NEW.km_inicio,
```

```
NEW.km_fin,  
  
NEW.sucursal_retiro,  
  
NEW.sucursal_devolucion,  
  
NEW.id_reserva,  
  
NEW.id_cliente,  
  
NEW.id_vehiculo,  
  
'I',  
  
v_usuario_app,  
  
v_usuario_db  
  
);
```

```
RETURN NEW;
```

```
-- UPDATE
```

```
ELSIF TG_OP = 'UPDATE' THEN
```

```
-- Antes update
```

```
INSERT INTO log_alquiler(  
  
    id_log,  
  
    id_alquiler,  
  
    fecha_inicio,  
  
    fecha_fin_prevista,  
  
    fecha_fin_real,
```

```
km_inicio,  
km_fin,  
sucursal_retiro,  
sucursal_devolucion,  
id_reserva,  
id_cliente,  
id_vehiculo,  
mov,  
usuario,  
usuario_db  
)  
VALUES(  
v_id_log,  
OLD.id_alquiler,  
OLD.fecha_inicio,  
OLD.fecha_fin_prevista,  
OLD.fecha_fin_real,  
OLD.km_inicio,  
OLD.km_fin,  
OLD.sucursal_retiro,  
OLD.sucursal_devolucion,  
OLD.id_reserva,  
OLD.id_cliente,
```



```
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES(  
    v_id_log,  
    NEW.id_alquiler,  
    NEW.fecha_inicio,  
    NEW.fecha_fin_prevista,  
    NEW.fecha_fin_real,  
    NEW.km_inicio,  
    NEW.km_fin,  
    NEW.sucursal_retiro,  
    NEW.sucursal_devolucion,  
    NEW.id_reserva,  
    NEW.id_cliente,  
    NEW.id_vehiculo,  
    'DU',  
    v_usuario_app,  
    v_usuario_db  
);  
  
RETURN NEW;
```

```
-- DELETE
```

```
ELSIF TG_OP = 'DELETE' THEN
```

```
    INSERT INTO log_alquiler(
```

```
        id_log,
```

```
        id_alquiler,
```

```
        fecha_inicio,
```

```
        fecha_fin_prevista,
```

```
        fecha_fin_real,
```

```
        km_inicio,
```

```
        km_fin,
```

```
        sucursal_retiro,
```

```
        sucursal_devolucion,
```

```
        id_reserva,
```

```
        id_cliente,
```

```
        id_vehiculo,
```

```
        mov,
```

```
        usuario,
```

```
        usuario_db
```

```
    )
```

```
VALUES(
```

```
    v_id_log,
```

```
        OLD.id_alquiler,  
        OLD.fecha_inicio,  
        OLD.fecha_fin_prevista,  
        OLD.fecha_fin_real,  
        OLD.km_inicio,  
        OLD.km_fin,  
        OLD.sucursal_retiro,  
        OLD.sucursal_devolucion,  
        OLD.id_reserva,  
        OLD.id_cliente,  
        OLD.id_vehiculo,  
        'D',  
        v_usuario_app,  
        v_usuario_db  
    );  
  
    RETURN OLD;  
  
END IF;  
  
RETURN NULL;  
  
END;
```

```
$$;  
  
CREATE TRIGGER trg_log_alquiler  
AFTER INSERT OR UPDATE OR DELETE  
ON alquiler  
FOR EACH ROW  
EXECUTE FUNCTION fn_log_alquiler();  
  
-- Alquiler_x_estado  
  
CREATE OR REPLACE FUNCTION fn_log_alquiler_x_estado()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
DECLARE  
  
    v_usuario_app INT;  
    v_usuario_db VARCHAR(50);  
    v_id_log INT;  
BEGIN  
  
    -- Usuario PostgreSQL  
    v_usuario_db := current_user;  
  
    -- Usuario Laravel
```

```
BEGIN

    v_usuario_app := current_setting('app.usuario', true)::INT;

EXCEPTION

    WHEN OTHERS THEN

        v_usuario_app := NULL;

END;

-- Obtener ID Log

v_id_log := nextval('seq_log_alquiler_x_estado');

-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_alquiler_x_estado(

        id_log,

        id_alquiler,

        id_estado_alquiler,

        fecha_estado,

        mov,

        usuario,

        usuario_db

    )

    VALUES(
```



```

        v_id_log,

        NEW.id_alquiler,

        NEW.id_estado_alquiler,

        NEW.fecha_estado,

        'I',

        v_usuario_app,

        v_usuario_db

    );

RETURN NEW;

-- UPDATE

ELSIF TG_OP = 'UPDATE' THEN

    -- Antes del update

    INSERT INTO log_alquiler_x_estado(

        id_log,

        id_alquiler,

        id_estado_alquiler,

        fecha_estado,

        mov,

        usuario,

        usuario_db
    )

```

```
RETURN NEW;
```

-- UPDATE

```
ELSIF TG_OP = 'UPDATE' THEN
```

-- Antes del update

```
INSERT INTO log_alquiler_x_estado(
    id_log,
    id_alquiler,
    id_estado_alquiler,
    fecha_estado,
    mov,
    usuario,
    usuario_db
```



UNIVERSIDAD
GASTÓN DACHARY

GASTÓN DACHARY

```

)

VALUES(

    v_id_log,

    OLD.id_alquiler,

    OLD.id_estado_alquiler,

    OLD.fecha_estado,

    'AU',

    v_usuario_app,

    v_usuario_db

);

-- Nuevo ID Log

v_id_log := nextval('seq_log_alquiler_x_estado');

-- Después del update

INSERT INTO log_alquiler_x_estado(

    id_log,

    id_alquiler,

    id_estado_alquiler,

    fecha_estado,

    mov,

    usuario,

    usuario_db

```

VALUES(

```
v_id_log,
```

```
OLD.id_alquiler,
```

```
OLD.id_estado_alquiler,
```

```
OLD.fecha_estado,
```

'AU',

```
v_usuario_app,
```

```
v_usuario_db
```

);

-- Nuevo ID Log

```
v_id_log := nextval('seq_log_alquiler_x_estado');
```

-- Después del update

```
INSERT INTO log_alquiler_x_estado(
```

```
id_log,
```

```
id_alquiler,
```

```
id_estado_alquiler,
```

```
fecha_estado,
```

mov,

```
usuario,
```

```
usuario_db
```




UNIVERSIDAD
GASTÓN DACHARY

```
)

VALUES(

    v_id_log,

    NEW.id_alquiler,

    NEW.id_estado_alquiler,

    NEW.fecha_estado,

    'DU',

    v_usuario_app,

    v_usuario_db

);

RETURN NEW;

-- DELETE

ELSIF TG_OP = 'DELETE' THEN

    INSERT INTO log_alquiler_x_estado(

        id_log,

        id_alquiler,

        id_estado_alquiler,

        fecha_estado,

        mov,

        usuario,
```

```
        usuario_db
    )
VALUES(
    v_id_log,
    OLD.id_alquiler,
    OLD.id_estado_alquiler,
    OLD.fecha_estado,
    'D',
    v_usuario_app,
    v_usuario_db
);

RETURN OLD;

END IF;

RETURN NULL;

END;
$$;

CREATE TRIGGER trg_log_alquiler_x_estado
AFTER INSERT OR UPDATE OR DELETE
```

```
ON alquiler_x_estado
FOR EACH ROW

EXECUTE FUNCTION fn_log_alquiler_x_estado();

-- Reserva_x_estado

CREATE OR REPLACE FUNCTION fn_log_reserva_x_estado()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE

    v_usuario_app INT;

    v_usuario_db VARCHAR(50);

    v_id_log INT;
BEGIN

    -- Usuario PostgreSQL

    v_usuario_db := current_user;

    -- Usuario Laravel

BEGIN

    v_usuario_app := current_setting('app.usuario', true)::INT;

EXCEPTION

    WHEN OTHERS THEN
```

```
v_usuario_app := NULL;

END;

-- ID Log

v_id_log := nextval('seq_log_reserva_x_estado');

-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_reserva_x_estado(

        id_log,

        id_reserva,

        id_estado_reserva,

        fecha_estado,

        mov,

        usuario,

        usuario_db

    )

    VALUES(

        v_id_log,

        NEW.id_reserva,

        NEW.id_estado_reserva,

        NEW.fecha_estado,
```



UNIVERSIDAD
GASTÓN DACHARY

```

        'I',

        v_usuario_app,

        v_usuario_db

    );

RETURN NEW;

-- UPDATE

ELSIF TG_OP = 'UPDATE' THEN

    -- Antes update

    INSERT INTO log_reserva_x_estado(

        id_log,

        id_reserva,

        id_estado_reserva,

        fecha_estado,

        mov,

        usuario,

        usuario_db

    )

    VALUES(

        v_id_log,

        OLD.id_reserva,

```

```
        OLD.id_estado_reserva,  
  
        OLD.fecha_estado,  
  
        'AU',  
  
        v_usuario_app,  
  
        v_usuario_db  
    );  
  
    -- Nuevo ID  
  
    v_id_log := nextval('seq_log_reserva_x_estado');  
  
    -- Después update  
  
    INSERT INTO log_reserva_x_estado(  
  
        id_log,  
  
        id_reserva,  
  
        id_estado_reserva,  
  
        fecha_estado,  
  
        mov,  
  
        usuario,  
  
        usuario_db  
    )  
  
    VALUES(  
  
        v_id_log,  
  
        NEW.id_reserva,
```

```
NEW.id_estado_reserva,  
  
NEW.fecha_estado,  
  
'DU',  
  
v_usuario_app,  
  
v_usuario_db  
  
);  
  
RETURN NEW;  
  
  
-- DELETE  
  
ELSIF TG_OP = 'DELETE' THEN  
  
INSERT INTO log_reserva_x_estado(  
  
    id_log,  
  
    id_reserva,  
  
    id_estado_reserva,  
  
    fecha_estado,  
  
    mov,  
  
    usuario,  
  
    usuario_db  
  
)  
  
VALUES(  
  
    v_id_log,
```

```
        OLD.id_reserva,  
        OLD.id_estado_reserva,  
        OLD.fecha_estado,  
        'D',  
        v_usuario_app,  
        v_usuario_db  
    );  
  
    RETURN OLD;  
  
END IF;  
  
RETURN NULL;  
  
END;  
$$;  
  
CREATE TRIGGER trg_log_reserva_x_estado  
AFTER INSERT OR UPDATE OR DELETE  
ON reserva_x_estado  
FOR EACH ROW  
EXECUTE FUNCTION fn_log_reserva_x_estado();
```



```
-- Vehiculo_x_estado

CREATE OR REPLACE FUNCTION fn_log_vehiculo_x_estado()
RETURNS TRIGGER

LANGUAGE plpgsql

AS $$

DECLARE

    v_usuario_app INT;

    v_usuario_db VARCHAR(50);

    v_id_log INT;

BEGIN

    -- Usuario PostgreSQL

    v_usuario_db := current_user;

    -- Usuario Laravel

    BEGIN

        v_usuario_app := current_setting('app.usuario', true)::INT;

    EXCEPTION

        WHEN OTHERS THEN

            v_usuario_app := NULL;

    END;

    -- ID Log
```

```
v_id_log := nextval('seq_log_vehiculo_x_estado');

-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_vehiculo_x_estado(

        id_log,

        id_veh_x_est,

        id_vehiculo,

        id_estado_vehiculo,

        fecha_estado,

        mov,

        usuario,

        usuario_db

    )

    VALUES(

        v_id_log,

        NEW.id_veh_x_est,

        NEW.id_vehiculo,

        NEW.id_estado_vehiculo,

        NEW.fecha_estado,

        'I',

        v_usuario_app,
```

```
v_usuario_db  
  
);  
  
RETURN NEW;  
  
-- UPDATE  
ELSIF TG_OP = 'UPDATE' THEN  
  
    -- Antes update  
    INSERT INTO log_vehiculo_x_estado(  
        id_log,  
        id_veh_x_est,  
        id_vehiculo,  
        id_estado_vehiculo,  
        fecha_estado,  
        mov,  
        usuario,  
        usuario_db  
    )  
    VALUES(  
        v_id_log,  
        OLD.id_veh_x_est,  
        OLD.id_vehiculo,
```

```
        OLD.id_estado_vehiculo,  
  
        OLD.fecha_estado,  
  
        'AU',  
  
        v_usuario_app,  
  
        v_usuario_db  
    );  
  
    -- Nuevo ID Log  
  
    v_id_log := nextval('seq_log_vehiculo_x_estado');  
  
    -- Después update  
  
    INSERT INTO log_vehiculo_x_estado(  
  
        id_log,  
  
        id_veh_x_est,  
  
        id_vehiculo,  
  
        id_estado_vehiculo,  
  
        fecha_estado,  
  
        mov,  
  
        usuario,  
  
        usuario_db  
    )  
  
    VALUES(  
  
        v_id_log,
```

```
NEW.id_veh_x_est,  
  
NEW.id_vehiculo,  
  
NEW.id_estado_vehiculo,  
  
NEW.fecha_estado,  
  
'DU',  
  
v_usuario_app,  
  
v_usuario_db  
  
);  
  
RETURN NEW;  
  
  
-- DELETE  
  
ELSIF TG_OP = 'DELETE' THEN  
  
INSERT INTO log_vehiculo_x_estado(  
  
    id_log,  
  
    id_veh_x_est,  
  
    id_vehiculo,  
  
    id_estado_vehiculo,  
  
    fecha_estado,  
  
    mov,  
  
    usuario,  
  
    usuario_db
```

```
)  
  
VALUES(  
  
    v_id_log,  
  
    OLD.id_veh_x_est,  
  
    OLD.id_vehiculo,  
  
    OLD.id_estado_vehiculo,  
  
    OLD.fecha_estado,  
  
    'D',  
  
    v_usuario_app,  
  
    v_usuario_db  
  
);  
  
    RETURN OLD;  
  
  
END IF;  
  
    RETURN NULL;  
  
END;  
$$;  
  
CREATE TRIGGER trg_log_vehiculo_x_estado  
AFTER INSERT OR UPDATE OR DELETE
```

```
ON vehiculo_x_estado
FOR EACH ROW

EXECUTE FUNCTION fn_log_vehiculo_x_estado();

-- Detalle_factura

CREATE OR REPLACE FUNCTION fn_log_detalle_factura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_usuario_app INT;
    v_usuario_db VARCHAR(50);
    v_id_log INT;
BEGIN

    -- Usuario PostgreSQL
    v_usuario_db := current_user;

    -- Usuario Laravel

BEGIN

    v_usuario_app := current_setting('app.usuario', true)::INT;

EXCEPTION

    WHEN OTHERS THEN
```

```
v_usuario_app := NULL;

END;

-- ID Log

v_id_log := nextval('seq_log_detalle_factura');

-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_detalle_factura(

        id_log,

        id_detalle_factura,

        codigo,

        descripcion,

        precio_unitario,

        subtotal,

        id_factura,

        mov,

        usuario,

        usuario_db

    )

    VALUES(

        v_id_log,
```




UNIVERSIDAD
GASTÓN DACHARY

```
NEW.id_detalle_factura,  
NEW.codigo,  
NEW.descripcion,  
NEW.precio_unitario,  
NEW.subtotal,  
NEW.id_factura,  
'I',  
v_usuario_app,  
v_usuario_db  
);  
  
RETURN NEW;  
  
-- UPDATE  
  
ELSIF TG_OP = 'UPDATE' THEN  
  
    -- Antes update  
  
    INSERT INTO log_detalle_factura(  
        id_log,  
        id_detalle_factura,  
        codigo,  
        descripcion,  
        precio_unitario,
```

```
        subtotal,  
        id_factura,  
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES(  
    v_id_log,  
    OLD.id_detalle_factura,  
    OLD.codigo,  
    OLD.descripcion,  
    OLD.precio_unitario,  
    OLD.subtotal,  
    OLD.id_factura,  
    'AU',  
    v_usuario_app,  
    v_usuario_db  
);  
  
-- Nuevo ID Log  
v_id_log := nextval('seq_log_detalle_factura');  
  
-- Después update
```

```
INSERT INTO log_detalle_factura(
```

```
    id_log,
```

```
    id_detalle_factura,
```

```
    codigo,
```

```
    descripcion,
```

```
    precio_unitario,
```

```
    subtotal,
```

```
    id_factura,
```

```
    mov,
```

```
    usuario,
```

```
    usuario_db
```

```
)
```

```
VALUES(
```

```
    v_id_log,
```

```
    NEW.id_detalle_factura,
```

```
    NEW.codigo,
```

```
    NEW.descripcion,
```

```
    NEW.precio_unitario,
```

```
    NEW.subtotal,
```

```
    NEW.id_factura,
```

```
    'DU',
```

```
    v_usuario_app,
```

```
    v_usuario_db
```

```
);

RETURN NEW;

-- DELETE

ELSIF TG_OP = 'DELETE' THEN

    INSERT INTO log_detalle_factura(

        id_log,

        id_detalle_factura,

        codigo,

        descripcion,

        precio_unitario,

        subtotal,

        id_factura,

        mov,

        usuario,

        usuario_db

    )

    VALUES(

        v_id_log,

        OLD.id_detalle_factura,

        OLD.codigo,
```

```
        OLD.descripcion,  
        OLD.precio_unitario,  
        OLD.subtotal,  
        OLD.id_factura,  
        'D',  
        v_usuario_app,  
        v_usuario_db  
    );  
  
    RETURN OLD;  
  
END IF;  
  
RETURN NULL;  
  
END;  
$$;  
  
CREATE TRIGGER trg_log_detalle_factura  
AFTER INSERT OR UPDATE OR DELETE  
ON detalle_factura  
FOR EACH ROW  
EXECUTE FUNCTION fn_log_detalle_factura();
```

```
-- Factura

CREATE OR REPLACE FUNCTION fn_log_factura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_usuario_app INT;
    v_usuario_db VARCHAR(50);
    v_id_log INT;
BEGIN

    -- Usuario PostgreSQL
    v_usuario_db := current_user;

    -- Usuario Laravel

BEGIN
    v_usuario_app := current_setting('app.usuario', true)::INT;
EXCEPTION
    WHEN OTHERS THEN
        v_usuario_app := NULL;
END;
```

```
-- ID Log

v_id_log := nextval('seq_log_factura');

-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_factura(

        id_log,

        id_factura,

        fecha_emision,

        monto_base,

        monto_recargo,

        total,

        id_alquiler,

        mov,

        usuario,

        usuario_db

    )

    VALUES(

        v_id_log,

        NEW.id_factura,

        NEW.fecha_emision,

        NEW.monto_base,
```



UNIVERSIDAD
GASTÓN DACHARY

```

        NEW.monto_recargo,

        NEW.total,

        NEW.id_alquiler,

        'I',

        v_usuario_app,

        v_usuario_db

    );

RETURN NEW;

-- UPDATE

ELSIF TG_OP = 'UPDATE' THEN

    -- Antes update

    INSERT INTO log_factura(

        id_log,

        id_factura,

        fecha_emision,

        monto_base,

        monto_recargo,

        total,

        id_alquiler,

        mov,

```



```
        usuario,  
        usuario_db  
    )  
VALUES(  
    v_id_log,  
    OLD.id_factura,  
    OLD.fecha_emision,  
    OLD.monto_base,  
    OLD.monto_recargo,  
    OLD.total,  
    OLD.id_alquiler,  
    'AU',  
    v_usuario_app,  
    v_usuario_db  
);  
  
-- Nuevo ID Log  
v_id_log := nextval('seq_log_factura');  
  
-- Después update  
INSERT INTO log_factura(  
    id_log,  
    id_factura,
```

```
        fecha_emision,  
        monto_base,  
        monto_recargo,  
        total,  
        id_alquiler,  
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES(  
    v_id_log,  
    NEW.id_factura,  
    NEW.fecha_emision,  
    NEW.monto_base,  
    NEW.monto_recargo,  
    NEW.total,  
    NEW.id_alquiler,  
    'DU',  
    v_usuario_app,  
    v_usuario_db  
);  
  
RETURN NEW;
```

```
-- DELETE
```

```
ELSIF TG_OP = 'DELETE' THEN
```

```
    INSERT INTO log_factura(
```

```
        id_log,
```

```
        id_factura,
```

```
        fecha_emision,
```

```
        monto_base,
```

```
        monto_recargo,
```

```
        total,
```

```
        id_alquiler,
```

```
        mov,
```

```
        usuario,
```

```
        usuario_db
```

```
    )
```

```
    VALUES(
```

```
        v_id_log,
```

```
        OLD.id_factura,
```

```
        OLD.fecha_emision,
```

```
        OLD.monto_base,
```

```
        OLD.monto_recargo,
```

```
        OLD.total,
```

```
        OLD.id_alquiler,  
        'D',  
        v_usuario_app,  
        v_usuario_db  
    );  
  
    RETURN OLD;  
  
END IF;  
  
RETURN NULL;  
  
END;  
$$;  
  
CREATE TRIGGER trg_log_factura  
AFTER INSERT OR UPDATE OR DELETE  
ON factura  
FOR EACH ROW  
EXECUTE FUNCTION fn_log_factura();  
  
-- Taller  
-- Función trigger
```

```
CREATE OR REPLACE FUNCTION fn_log_taller()
RETURNS TRIGGER

LANGUAGE plpgsql

AS $$

DECLARE

    v_usuario_app INT;

    v_usuario_db VARCHAR(50);

    v_id_log INT;

BEGIN

    -- Usuario PostgreSQL

    v_usuario_db := current_user;

    -- Usuario Laravel

    BEGIN

        v_usuario_app := current_setting('app.usuario', true)::INT;

    EXCEPTION

        WHEN OTHERS THEN

            v_usuario_app := NULL;

    END;

    -- ID Log

    v_id_log := nextval('seq_log_taller');
```

```
-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_taller(

        id_log,

        id_taller,

        nombre_taller,

        direccion_taller,

        telefono_taller,

        id_departamento,

        mov,

        usuario,

        usuario_db

    )

    VALUES(

        v_id_log,

        NEW.id_taller,

        NEW.nombre_taller,

        NEW.direccion_taller,

        NEW.telefono_taller,

        NEW.id_departamento,

        'I',
```

```
        v_usuario_app,  
        v_usuario_db  
    );  
  
    RETURN NEW;  
  
-- UPDATE  
ELSIF TG_OP = 'UPDATE' THEN  
  
    -- Antes update  
    INSERT INTO log_taller(  
        id_log,  
        id_taller,  
        nombre_taller,  
        direccion_taller,  
        telefono_taller,  
        id_departamento,  
        mov,  
        usuario,  
        usuario_db  
    )  
    VALUES(  
        v_id_log,
```

```
        OLD.id_taller,  
  
        OLD.nombre_taller,  
  
        OLD.direccion_taller,  
  
        OLD.telefono_taller,  
  
        OLD.id_departamento,  
  
        'AU',  
  
        v_usuario_app,  
  
        v_usuario_db  
    );  
  
    -- Nuevo ID Log  
  
    v_id_log := nextval('seq_log_taller');  
  
    -- Después update  
  
    INSERT INTO log_taller(  
  
        id_log,  
  
        id_taller,  
  
        nombre_taller,  
  
        direccion_taller,  
  
        telefono_taller,  
  
        id_departamento,  
  
        mov,  
  
        usuario,
```



```
        usuario_db
    )
VALUES(
    v_id_log,
    NEW.id_taller,
    NEW.nombre_taller,
    NEW.direccion_taller,
    NEW.telefono_taller,
    NEW.id_departamento,
    'DU',
    v_usuario_app,
    v_usuario_db
);

RETURN NEW;

-- DELETE

ELSIF TG_OP = 'DELETE' THEN

    INSERT INTO log_taller(
        id_log,
        id_taller,
        nombre_taller,
```

```
        direccion_taller,  
        telefono_taller,  
        id_departamento,  
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES(  
    v_id_log,  
    OLD.id_taller,  
    OLD.nombre_taller,  
    OLD.direccion_taller,  
    OLD.telefono_taller,  
    OLD.id_departamento,  
    'D',  
    v_usuario_app,  
    v_usuario_db  
);  
  
RETURN OLD;  
  
END IF;
```

```
RETURN NULL;

END;

$$;

CREATE TRIGGER trg_log_taller
AFTER INSERT OR UPDATE OR DELETE
ON taller
FOR EACH ROW
EXECUTE FUNCTION fn_log_taller();

-----

-- Mantenimiento
-- Función trigger

CREATE OR REPLACE FUNCTION fn_log_mantenimiento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_usuario_app INT;
    v_usuario_db VARCHAR(50);
    v_id_log INT;
BEGIN
```

```
-- Usuario PostgreSQL

v_usuario_db := current_user;

-- Usuario Laravel

BEGIN

    v_usuario_app := current_setting('app.usuario', true)::INT;

EXCEPTION

    WHEN OTHERS THEN

        v_usuario_app := NULL;

END;

-- ID Log

v_id_log := nextval('seq_log_mantenimiento');

-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_mantenimiento(

        id_log,

        id_mantenimiento,

        fecha_envio,

        fecha_devolucion,
```

```
        id_vehiculo,  
        id_taller,  
        mov,  
        usuario,  
        usuario_db  
    )  
VALUES(  
    v_id_log,  
    NEW.id_mantenimiento,  
    NEW.fecha_envio,  
    NEW.fecha_devolucion,  
    NEW.id_vehiculo,  
    NEW.id_taller,  
    'I',  
    v_usuario_app,  
    v_usuario_db  
);  
  
RETURN NEW;  
  
-- UPDATE  
ELSIF TG_OP = 'UPDATE' THEN
```

```
-- Antes update

INSERT INTO log_mantenimiento(

    id_log,

    id_mantenimiento,

    fecha_envio,

    fecha_devolucion,

    id_vehiculo,

    id_taller,

    mov,

    usuario,

    usuario_db

)

VALUES(

    v_id_log,

    OLD.id_mantenimiento,

    OLD.fecha_envio,

    OLD.fecha_devolucion,

    OLD.id_vehiculo,

    OLD.id_taller,

    'AU',

    v_usuario_app,

    v_usuario_db

);
```

```
-- Nuevo ID Log

v_id_log := nextval('seq_log_mantenimiento');

-- Después update

INSERT INTO log_mantenimiento(

    id_log,

    id_mantenimiento,

    fecha_envio,

    fecha_devolucion,

    id_vehiculo,

    id_taller,

    mov,

    usuario,

    usuario_db

)

VALUES(

    v_id_log,

    NEW.id_mantenimiento,

    NEW.fecha_envio,

    NEW.fecha_devolucion,

    NEW.id_vehiculo,

    NEW.id_taller,
```

```
'DU',  
  
v_usuario_app,  
  
v_usuario_db  
  
);  
  
RETURN NEW;  
  
  
-- DELETE  
  
ELSIF TG_OP = 'DELETE' THEN  
  
    INSERT INTO log_mantenimiento(  
  
        id_log,  
  
        id_mantenimiento,  
  
        fecha_envio,  
  
        fecha_devolucion,  
  
        id_vehiculo,  
  
        id_taller,  
  
        mov,  
  
        usuario,  
  
        usuario_db  
  
    )  
  
    VALUES(  
  
        v_id_log,
```



```
        OLD.id_mantenimiento,  
  
        OLD.fecha_envio,  
  
        OLD.fecha_devolucion,  
  
        OLD.id_vehiculo,  
  
        OLD.id_taller,  
  
        'D',  
  
        v_usuario_app,  
  
        v_usuario_db  
    );  
  
    RETURN OLD;  
  
END IF;  
  
RETURN NULL;  
  
END;  
$$;  
  
-- Trigger  
  
CREATE TRIGGER trg_log_mantenimiento  
AFTER INSERT OR UPDATE OR DELETE  
ON mantenimiento
```

```
FOR EACH ROW

EXECUTE FUNCTION fn_log_mantenimiento();

-----

-- tarifa

CREATE OR REPLACE FUNCTION fn_log_tarifa()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE

    v_usuario_app INT;

    v_usuario_db VARCHAR(50);

    v_id_log INT;
BEGIN

    -- Usuario PostgreSQL

    v_usuario_db := current_user;

    -- Usuario Laravel

BEGIN

    v_usuario_app := current_setting('app.usuario', true)::INT;

EXCEPTION

    WHEN OTHERS THEN
```

```
v_usuario_app := NULL;

END;

-- ID Log

v_id_log := nextval('seq_log_tarifa');

-- INSERT

IF TG_OP = 'INSERT' THEN

    INSERT INTO log_tarifa(

        id_log,

        id_tarifa,

        precio_dia,

        porcentaje_recargo_hora,

        id_sucursal,

        id_tipo_vehiculo,

        mov,

        usuario,

        usuario_db

    )

    VALUES(

        v_id_log,

        NEW.id_tarifa,
```



```
NEW.precio_dia,  
NEW.porcentaje_recargo_hora,  
NEW.id_sucursal,  
NEW.id_tipo_vehiculo,  
'I',  
v_usuario_app,  
v_usuario_db  
);
```

```
RETURN NEW;
```

-- UPDATE

```
ELSIF TG_OP = 'UPDATE' THEN
```

-- Antes update

```
INSERT INTO log_tarifa(
    id_log,
    id_tarifa,
    precio_dia,
    porcentaje_recargo_hora,
    id_sucursal,
    id_tipo_vehiculo,
    mov,
```

```
        usuario,  
        usuario_db  
    )  
VALUES(  
    v_id_log,  
    OLD.id_tarifa,  
    OLD.precio_dia,  
    OLD.porcentaje_recargo_hora,  
    OLD.id_sucursal,  
    OLD.id_tipo_vehiculo,  
    'AU',  
    v_usuario_app,  
    v_usuario_db  
);  
  
-- Nuevo ID Log  
v_id_log := nextval('seq_log_tarifa');  
  
-- Después update  
INSERT INTO log_tarifa(  
    id_log,  
    id_tarifa,  
    precio_dia,
```



UNIVERSIDAD
GASTÓN DACHARY

```

        porcentaje_recargo_hora,

        id_sucursal,

        id_tipo_vehiculo,

        mov,

        usuario,

        usuario_db

    )

VALUES(

    v_id_log,

    NEW.id_tarifa,

    NEW.precio_dia,

    NEW.porcentaje_recargo_hora,

    NEW.id_sucursal,

    NEW.id_tipo_vehiculo,

    'DU',

    v_usuario_app,

    v_usuario_db

);

RETURN NEW;

-- DELETE

ELSIF TG_OP = 'DELETE' THEN

```

```
INSERT INTO log_tarifa(  
    id_log,  
    id_tarifa,  
    precio_dia,  
    porcentaje_recargo_hora,  
    id_sucursal,  
    id_tipo_vehiculo,  
    mov,  
    usuario,  
    usuario_db  
)  
VALUES(  
    v_id_log,  
    OLD.id_tarifa,  
    OLD.precio_dia,  
    OLD.porcentaje_recargo_hora,  
    OLD.id_sucursal,  
    OLD.id_tipo_vehiculo,  
    'D',  
    v_usuario_app,  
    v_usuario_db  
);
```

```
        RETURN OLD;

    END IF;

    RETURN NULL;

END;
$$;

CREATE TRIGGER trg_log_tarifa
AFTER INSERT OR UPDATE OR DELETE
ON tarifa
FOR EACH ROW
EXECUTE FUNCTION fn_log_tarifa();
```

Esquema Logs

```
CREATE TABLE log_sucursal (

    id_log INT PRIMARY KEY,

    id_sucursal INT,

    nombre_sucursal VARCHAR(50) NOT NULL,

    direccion_sucursal VARCHAR(100),

    id_departamento INT,

    mov varchar(2),
```



```
fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

usuario int,

usuario_db VARCHAR(50)

);

CREATE TABLE log_tipo_vehiculo (

    id_log INT PRIMARY KEY,

    id_tipo_vehiculo INT,

    nombre_tipo_vehiculo VARCHAR(50) NOT NULL,

    mov varchar(2),

    fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    usuario int,

    usuario_db VARCHAR(50)

);

CREATE TABLE log_taller (

    id_log SERIAL PRIMARY KEY,

    id_taller INT,

    nombre_taller VARCHAR(50) NOT NULL,

    direccion_taller VARCHAR(100),

    telefono_taller VARCHAR(30),

    id_departamento INT,

    mov varchar(2),
```

```
fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
usuario int,  
  
usuario_db VARCHAR(50)  
);  
  
CREATE TABLE log_vehiculo (  
  
    id_log SERIAL PRIMARY KEY,  
  
    id_vehiculo INT,  
  
    patente VARCHAR(10),  
  
    marca VARCHAR(20),  
  
    modelo VARCHAR(20),  
  
    detalle_confort VARCHAR(500),  
  
    id_tipo_vehiculo INT,  
  
    id_sucursal INT,  
  
    mov varchar(2),  
  
    fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    usuario int,  
  
    usuario_db VARCHAR(50)  
);  
  
CREATE TABLE log_mantenimiento (  
  
    id_log SERIAL PRIMARY KEY,
```

```
id_mantenimiento INT,  
fecha_envio DATE,  
fecha_devolucion DATE,  
id_vehiculo INT,  
id_taller INT,  
mov varchar(2),  
fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
usuario int,  
usuario_db VARCHAR(50)  
);  
  
CREATE TABLE log_tarifa (  
    id_log SERIAL PRIMARY KEY,  
    id_tarifa INT,  
    precio_dia NUMERIC(10,2),  
    porcentaje_recargo_hora NUMERIC(10,2),  
    id_sucursal INT,  
    id_tipo_vehiculo INT,  
    mov varchar(2),  
    fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    usuario int,  
    usuario_db VARCHAR(50)  
);
```

```
CREATE TABLE log_reserva (  
    id_log SERIAL PRIMARY KEY,  
    id_reserva INT,  
    fecha_inicio TIMESTAMP NOT NULL,  
    fecha_fin TIMESTAMP NOT NULL,  
    sucursal_retiro INT NOT NULL,  
    sucursal_devolucion INT,  
    id_cliente INT NOT NULL,  
    id_vehiculo INT NOT NULL,  
    mov varchar(2),  
    fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    usuario int,  
    usuario_db VARCHAR(50)  
);
```

```
CREATE TABLE log_alquiler (  
    id_log SERIAL PRIMARY KEY,  
    id_alquiler INT,  
    fecha_inicio TIMESTAMP NOT NULL,  
    fecha_fin_prevista TIMESTAMP NOT NULL,  
    fecha_fin_real TIMESTAMP,  
    km_inicio INT NOT NULL,
```

```
km_fin INT,  
sucursal_retiro INT NOT NULL,  
sucursal_devolucion INT,  
id_reserva INT,  
id_cliente INT,  
id_vehiculo INT,  
mov varchar(2),  
fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
usuario int,  
usuario_db VARCHAR(50)  
);  
  
CREATE TABLE log_factura (  
    id_log SERIAL PRIMARY KEY,  
    id_factura INT,  
    fecha_emision TIMESTAMP NOT NULL,  
    monto_base NUMERIC(10,2),  
    monto_recargo NUMERIC(10,2),  
    total NUMERIC(10,2),  
    id_alquiler INT NOT NULL,  
    mov varchar(2),  
    fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    usuario int,
```

```
usuario_db VARCHAR(50)

);

CREATE TABLE log_detalle_factura (

    id_log SERIAL PRIMARY KEY,

    id_detalle_factura INT,

    codigo INT,

    descripcion VARCHAR(100),

    precio_unitario NUMERIC(10,2),

    subtotal NUMERIC(10,2),

    id_factura INT,

    mov varchar(2),

    fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    usuario int,

    usuario_db VARCHAR(50)

);

CREATE TABLE log_alquiler_x_estado (

    id_log INT,

    id_alquiler INT NOT NULL,

    id_estado_alquiler INT NOT NULL,

    fecha_estado TIMESTAMP NOT NULL,

    mov varchar(2),
```

```
fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
usuario int,  
  
usuario_db VARCHAR(50)  
);  
  
CREATE TABLE log_reserva_x_estado (  
  
    id_log SERIAL PRIMARY KEY,  
  
    id_reserva INT,  
  
    id_estado_reserva INT,  
  
    fecha_estado TIMESTAMP NOT NULL,  
  
    mov varchar(2),  
  
    fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    usuario int,  
  
    usuario_db VARCHAR(50)  
);  
  
CREATE TABLE log_vehiculo_x_estado (  
  
    id_log SERIAL PRIMARY KEY,  
  
    id_veh_x_est INT,  
  
    id_vehiculo INT NOT NULL,  
  
    id_estado_vehiculo INT NOT NULL,  
  
    fecha_estado TIMESTAMP,  
  
    mov varchar(2),
```

```
fecha_mov TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
usuario int,  
  
usuario_db VARCHAR(50)  
);
```

Jobs

```
CREATE OR REPLACE FUNCTION fn_alquileres_vencidos()  
  
RETURNS TABLE (  
  
    id_alquiler INT,  
  
    id_vehiculo INT,  
  
    patente VARCHAR,  
  
    fecha_fin_prevista TIMESTAMP,  
  
    horas_atraso NUMERIC  
  
)  
  
LANGUAGE plpgsql  
  
AS $$  
  
BEGIN  
  
    RETURN QUERY  
  
    SELECT  
  
        a.id_alquiler,  
  
        v.id_vehiculo,  
  
        v.patente,  
  
        a.fecha_fin_prevista,
```



```
ROUND(  
    EXTRACT(EPOCH FROM (  
        CURRENT_TIMESTAMP - a.fecha_fin_prevista  
    )) / 3600,  
    2  
    ) AS horas_atraso  
  
FROM alquiler a  
INNER JOIN vehiculo v  
    ON v.id_vehiculo = a.id_vehiculo  
  
WHERE a.fecha_fin_prevista < CURRENT_TIMESTAMP  
AND a.fecha_fin_real IS NULL  
  
AND (  
    SELECT axe.id_estado_alquiler  
    FROM alquiler_x_estado axe  
    WHERE axe.id_alquiler = a.id_alquiler  
    ORDER BY axe.fecha_estado DESC  
    LIMIT 1  
    ) = 1;
```

END;

\$\$;